Blind Adaptor Signatures, Revisited: Stronger Security Definitions and Their Construction toward Practical Applications

Masashi Hisai
Panasonic Holdings Corporation
1006 Kadoma, Kadoma City, Osaka, Japan
hisai.masashi@jp.panasonic.com

Naoto Yanai
Panasonic Holdings Corporation
1006 Kadoma, Kadoma City, Osaka, Japan
yanai.naoto@jp.panasonic.com

Abstract

Although both blind signatures and adaptor signatures have individually attracted attention, there is little research on combining these primitives so far. To the best of our knowledge, although the only existing scheme is the scheme by Qin et al. (S&P 2023), it does not consider practical security notions, namely full extractability, unlinkability, and pre-verify soundness, especially against adversaries with rich attack interfaces. In this paper, we propose the first blind adaptor signature scheme that satisfies the above security definitions. We first formalize the security of a blind adaptor signature scheme and prove a relationship between our security definitions and the existing security definitions, as well as showing several gaps in the existing schemes as a technical problem. Our main idea to overcome this problem is to leverage relations that support random self-reducibility instead of additional random numbers for blind signatures. Such a construction can embed relations into the signature components by re-randomizing them with the relations, and hence satisfies all the above security definitions. We then introduce new proof techniques to prove the full extractability by leveraging the unlinkability. We also discuss applications of the proposed scheme.

1 Introduction

1.1 Background

Adaptor signatures (ASes) [1] are digital signatures that provide witnesses for statements in NP relations as secret information, as well as providing the signature generation defined on standard signatures, such as the Schnorr signatures [2]. Loosely speaking, a signer (also called sender) generates pre-signatures on statements of some relation and messages, and then a user (also called receiver) generates signatures (called adapted signatures) on a standard signature scheme, e.g., the Schnorr signatures, from the pre-signatures by witnesses of the statements. In doing so, the signer can extract the witnesses from the adapted signatures and the pre-signatures, in agreement with the user, and both signatures can be verified with a common verification key. ASes enable us to provide fair trading between a sender and a receiver on the Internet using only digital signatures. For example, ASes can realize "scriptless scripts" that provide complicated trading on blockchains without smart contracts [1]. These can also improve the scalability problem of blockchains [3, 4] and interoperability [5, 6] through a design of atomic swap between different blockchains [7].

In this paper, we focus on (weak) blindness of ASes whereby a signer signs messages without knowing them. Since information on trading often contains users' sensitive preferences and lifestyles, it is desirable to guarantee the blindness of messages corresponding to the trading [8, 9]. *Blind signatures* (BSes) [10] are a well-known signature scheme for the above blindness, and Qin et al. [11] proposed the first *blind adaptor signature* (BAS) scheme that combines ASes with BSes. The original motivation of Qin et al. was to design blind payment channels [11] for blockchains, and there are also applications, such as fair trading for electronic resources [12].

However, to the best of our knowledge, the only existing scheme by Qin et al. depends on the first security definition by Aumayr et al. [3]. Namely, it discussed neither full extractability [13], unlinkability [13], nor pre-verify soundness [14], and is also unable to represent rich attack interfaces of an adversary differently from the state-of-the-art BSes [15]. The above fact may be a serious issue for an application of BASes. First, unless the full extractability is satisfied, an adversary can perform trading without agreement with a signer because of forging pre-signatures whose witnesses cannot be extracted. Second, unless the unlinkability is satisfied, an adversary can breach information on trading from messages corresponding to their signatures. Third, unless the pre-verify soundness is satisfied, an adversary can trick a verifier into accepting pre-signatures that cannot be adapted even with valid witnesses. Dai et al. [13] and Gehart et al. [14] point out that security gaps exist when ASes do not satisfy the above properties. That is, satisfying the above three definitions is crucial for various practical applications. The goal of this paper is to formalize the above three definitions for BASes, including an adversary who can utilize rich attack interfaces, and then propose a novel BAS scheme with its security proofs.

1.2 Technical Problem

Constructing a (predicate) BAS (PBAS) scheme under the above motivation is *non-trivial*. As ASes are defined based on standard digital signatures [3, 13], unlinkability guarantees that their adapted signatures are indistinguishable from the standard digital signatures as described above. Since the Schnorr signatures have been utilized as standard signatures on major blockchains, such as Bitcoin [16] and Ethereum [1, 17], we are motivated to design a BAS scheme whose algebraic structures are identical to Schnorr signatures for the unlinkability. Thus, the unlinkability restricts possible algebraic structures of BASes, since it requires preserving the structure of Schnorr signatures. Likewise, BSes often break algebraic structures to satisfy blindness of messages, e.g., the use of hash functions or providing additional random numbers. In contrast, the full extractability and the pre-verify soundness need an attractive algebraic structure, such as a homomorphism, to compute relations between statements and their witnesses through signatures. Namely, the main technical problem is to satisfy full extractability and pre-verify soundness for ASes under the algebraic structure of Schnorr signatures, which is limited due to unlinkability and (weak) blindness. Indeed, we found a simple gap where the scheme by Qin et al. does not satisfy the unlinkability in Section 3.2.1. We also need to define security against

a stronger adversary than that by Qin et al. in this paper in order to represent rich attack interfaces of the state-of-the-art BSes [15].

One might think that blind Schnorr signatures [15] can be extended into BASes by the existing conversions [13, 14] from standard signatures. However, it is not straightforward. Whereas ASes need algebraic structures that enable a user to embed a witness into any pre-signatures and a signer to extract the witness with its adapted signature, BSes often break algebraic structures for the blindness of messages. We show a gap where the existing conversion [14] of some BSes [15] does not satisfy the full extractability in Section 3.2.1. This means that the full extractability may contradict the blindness.

1.3 Our Contributions

In this paper, we propose the first PBAS scheme, named PBASch for the sake of convenience, that satisfies the full extractability, the unlinkability, and the pre-verify soundness as well as the existing security definition of BASes [11]. To this end, we also formally define these security notions. Our main idea to tackle the above technical problem is to leverage relations that support the strong random self-reducibility (SRSR) [13], where both statements and witnesses become indistinguishable from independently sampled statements and witnesses: for instance, we utilize these relations under some homomorphism of the Schnorr signatures instead of additional random numbers to break algebraic structures in blind signatures. It will realize a (P)BAS scheme that satisfies the full extractability, the unlinkability, and the pre-verify soundness against adversaries with rich attack interfaces and weak blindness. We also discuss the performance evaluation of PBASch and its applications in the real world. We describe their details below.

Formal Security Definitions. We formally define the security of BASes, including the full extractability, the unlinkability, and the pre-verify soundness as well as introducing rich attack interfaces for the state-of-the-art BSes [15] into the existing definitions by Qin et al. [11]. Although it is different from our main motivation, we also formalize witness hiding [18]. Furthermore, inspired by Dai et al. [13], we prove that our full extractability implies the existing definitions by Qin et al., but not vice versa. While Dai et al. proved a similar relationship for (non-blind) ASes, it is different from our definitions and proofs because of the functionality of BASes.

Design of (Predicate) Blind Adaptor Signatures. We propose the PBAS scheme PBASch as the main contribution. In PBASch, signatures are decomposed into the randomness component and the message component through a Schnorr-based construction, i.e., $(R, s) \in \mathbb{G} \times \mathbb{Z}_q$. These components are re-randomized with relations to support SRSR: for instance, the randomness component R is re-randomized by a statement $Y \in \mathbb{G}$ and the message component s is re-randomized by a witness $y \in \mathbb{Z}_q$ as an element in each group, respectively. The above construction can keep the same distribution as the Schnorr signatures because of re-randomizing their component. This means that PBASch satisfies unlinkability as well as the weak blindness. It also enables us to maintain the algebraic structure with homomorphism and therefore can support the full extractability. We can simultaneously introduce a membership test for the pre-verify soundness. We also introduce new proof techniques for the full extractability. In a nutshell, the full extractability is reduced to the unforgeability of the Schnorr signature scheme, the soundness of the non-interactive zero-knowledge argument scheme, and the one-wayness of the discrete logarithm problem via the above unlinkability. The above techniques indicate that SRSR is essential for BASes. We also note that PBASch is a predicate construction and satisfies the witness hiding.

Table 1: Comparison of the proposed scheme with the existing works.

	Qin et al. [11]	Gerhart et al. [14]	Fuchsbauer et al. [15]	Proposed Scheme
Type of Scheme	blind adaptor	adaptor	blind	blind adaptor
Weak Blindness	✓		✓	✓
Unlinkability		✓		✓
Full Extractability		✓		✓
Witness Hiding		✓		✓
Pre-Verify Soundness		✓		✓
Predicates			✓	✓

Applications. We also discuss privacy-enhancing applications of the proposed scheme. The proposed scheme can potentially improve the fungibility of cryptocurrency of atomic swaps [19] as well as providing stronger security in pre-scheduled payments [20] by virtue of the full extractability, the unlinkability, and the pre-verify soundness.

1.4 Related Works

After Aumayr et al. [3] formalized the security of ASes, many security notions were defined, i.e., full extractability [13], unlinkability [13], pre-verify soundness [14], and witness hiding [18, 21]. There are also conversions from an identification scheme [22] and from standard signatures with some conditions [13, 14]. Indeed, there exist adaptor signatures that can be used for any NP relation from any one-way function [18, 23]. Although the above conversions are potential approaches, there is an important gap because they do not cover BSes. BSes have been extended in various ways [24, 25, 26], including predicate construction [15]. Our proposed scheme is inspired by the scheme by Fuchsbauer et al. [15] in terms of a predicate construction.

Although there are also many works for the design of ASes [27, 28] and BSes [29, 30, 31, 32, 33], BASes have not been discussed except for Qin et al. [11], to the best of our knowledge. Although Qin et al. discussed neither full extractability nor unlinkability [13], both their BAS scheme and blind payment channels are elegant. We are motivated by improving blind payment channels through the design of BASes. The theoretical comparison is shown in Table 1. We note that our proposed scheme also satisfies the witness hiding [18] since it is implied from the full extractability. Although there are stronger variations of witness hiding [21], we leave it as an open problem to construct a (P)BAS scheme that satisfies them.

As a privacy-enhancing extension of adaptor signatures, user anonymity [34] based on linkable ring signatures [35] has been

discussed to construct payment channels for Monero[1]. However, it strongly depends on Monero and its underlying signatures [36]. In contrast, we discuss blindness of messages through a construction based on the Schnorr signatures, which are applicable to Bitcoin [16] and Ethereum [1, 17] as described above. We also describe further related works in Appendix A due to the page limitation.

2 Preliminaries

In this section, we describe several mathematical notions related to the full extractability and the unlinkability in this paper, and the figures are described in Appendix B due to the page limitation. The other notions are presented in Appendix C.

2.1 Discrete Logarithm Problem

A group generation algorithm GrGen is a probabilistic polynomial time algorithm. GrGen takes a security parameter 1^λ as input and then outputs a group description $\Gamma = (q, \mathbb{G}, G)$, where $\mathbb{G}$ is a group of prime order q and G is a generator of $\mathbb{G}$.

Definition 2.1 (Discrete Logarithm). We say that a group generation algorithm GrGen satisfies discrete-logarithm (DL) hardness if, for any probabilistic polynomial time adversary $\mathcal{A}$, the probability $\mathrm{Adv}^{\mathrm{DL}}_{\mathrm{GrGen},\mathcal{A}}(\lambda) = \Pr[\mathrm{DL}^{\mathcal{A}}_{\mathrm{GrGen}}(\lambda) = 1]$ that $\mathcal{A}$ wins the game shown in Fig. 12 is negligible for a security parameter λ.

2.2 Relations

If some relation $\mathrm{R} \subseteq \{0,1\}^* \times \{0,1\}^*$ is an NP relation, there exists a polynomial time algorithm to verify $(Y, y) \in \mathrm{R}$. Then, a language $\mathbb{L}_\mathrm{R}$ for a relation R is defined as $\mathbb{L}_\mathrm{R} = \{Y \in \{0,1\}^* \mid \exists y \in \{0,1\}^* : (Y, y) \in \mathrm{R}\}$. There then exists a polynomial time algorithm $(Y, y) \leftarrow \mathrm{R.Gen}(1^\lambda)$ to generate a statement Y and a witness y for R.

Definition 2.2 (SRSR Relation). We say that a relation R is strongly random self-reducible (SRSR) if there exists some set $\mathrm{R.R}_\lambda$ for a relation R and if the probability $\mathrm{Adv}^{\mathrm{SRSR}}_{\mathrm{R,A}}(\lambda) := \left|\Pr[\mathrm{SRSR}^{\mathcal{A},0}_\mathrm{R}] - \Pr[\mathrm{SRSR}^{\mathcal{A},1}_\mathrm{R}]\right|$ that the probabilistic polynomial time adversary $\mathcal{A}$ wins the game shown in Fig. 1 is negligible for a security parameter λ and $b \leftarrow \{0,1\}$. Here, suppose that there exists polynomial time algorithms R.A, R.B, R.C for R with the following conditions for any $r \leftarrow \mathrm{R.R}_\lambda$ and $(Y, y) \in \mathrm{R.Gen}(1^\lambda)$: for $Y' \leftarrow \mathrm{R.A}(Y, r)$, the distribution of Y' is identical to Y generated from $\mathrm{R.Gen}(1^\lambda)$; for $Y' \leftarrow \mathrm{R.A}(Y, r)$ and $y' \leftarrow \mathrm{R.B}(y, r)$, $(Y', y') \in \mathrm{R}$ holds; and, for $y \leftarrow \mathrm{R.C}(y', r)$ such that $y' \leftarrow \mathrm{R.B}(y, r)$, $(Y, y) \in \mathrm{R}$ holds.

$\mathrm{SRSR}^{\mathcal{A},b}_\mathrm{R}(\lambda)$
$b' \leftarrow \mathcal{A}^{\mathrm{New}}()$
if $b = b'$ **return** 1
else return 0

$\mathrm{New}(Y, y)$
if $(Y, y) \notin R$ **then return** $\perp$
$(Y', y') \leftarrow \mathrm{R.Gen}(1^\lambda)$; $r \leftarrow \mathrm{R.R}_\lambda$
$Y_0 \leftarrow \mathrm{R.A}(Y, r)$; $y_0 \leftarrow \mathrm{R.B}(y, r)$
$Y_1 \leftarrow \mathrm{R.A}(Y', r)$; $y_1 \leftarrow \mathrm{R.B}(y', r)$
return (Y_b, y_b)

Figure 1: The SRSR game.

Definition 2.3 (q-OW). We say that a relation R is q-one-wayness (q-OW) if, for any probabilistic polynomial time adversary $\mathcal{A}$, the probability $\mathrm{Adv}^{\mathrm{q-OW}}_{\mathrm{R,A}}(\lambda) = \Pr[\mathrm{qOW}^{\mathcal{A}}_\mathrm{R}(\lambda) = 1]$ that $\mathcal{A}$ wins the game shown in Fig. 13 is negligible for a security parameter λ.

2.3 Digital Signatures

A digital signature scheme Sig consists of the following algorithms: (1) $sp \leftarrow \mathrm{Setup}(1^\lambda)$: The setup algorithm takes a security parameter 1^λ as input and then outputs a public parameter sp; (2) $(sk, vk) \leftarrow \mathrm{KeyGen}(sp)$: The key generation algorithm takes a public parameter sp as input and then outputs a signing key sk and a verification key vk; (3) $\sigma \leftarrow \mathrm{Sign}(sk, m)$: The signing algorithm takes a signing key sk and a message m as input and then outputs a signature σ; (4) $b \leftarrow \mathrm{Verify}(vk, m, \sigma)$: The verification algorithm takes a verification key vk, a message m and a signature σ as input, and then outputs $b = 1$ or $b = 0$. We say that a digital signature scheme Sig satisfies correctness if, for any m and λ, $\Pr[\mathrm{Verify}(vk, m, \mathrm{Sign}(sk, m)) = 1] = 1$ holds for any $(sk, vk) \leftarrow \mathrm{KeyGen}(sp)$ with respect to $sp \leftarrow \mathrm{Setup}(1^\lambda)$.

Definition 2.4 (sEUF-CMA). We say that a digital signature scheme Sig satisfies strongly existential unforgeability against chosen message attacks (sEUF-CMA) if, for any probabilistic polynomial time adversary $\mathcal{A}$, the probability $\mathrm{Adv}^{\mathrm{CMA}}_{\mathrm{Sig},\mathcal{A}}(\lambda) = \Pr[\mathrm{sEUF}^{\mathcal{A}}_{\mathrm{Sig}}(\lambda) = 1]$ that $\mathcal{A}$ wins the game shown in Fig. 14 is negligible for a security parameter λ.

Schnorr Signatures. The Schnorr signature scheme [2] Sch based on a group generation algorithm GrGen is a digital signature scheme under the DL hardness. It is constructed as follows: (1) $sp \leftarrow \mathrm{Setup}(1^\lambda)$: The setup algorithm takes a security parameter 1^λ as input and then outputs a set of parameters $sp := (q, \mathbb{G}, G, \mathrm{H})$, where $(q, \mathbb{G}, G) \leftarrow \mathrm{GrGen}(1^\lambda)$ and $\mathrm{H} : \{0,1\}^* \to \mathbb{Z}_q$ is a hash function. (2) $(sk, vk) \leftarrow \mathrm{KeyGen}(sp)$: The key generation algorithm takes a set of parameters $sp = (q, \mathbb{G}, G, \mathrm{H})$ as input. It generates $x \leftarrow \mathbb{Z}_q$, and computes $X = xG$. It then outputs a signing key $sk = (sp, x)$ and a verification key $vk = (sp, X)$. (3) $\sigma \leftarrow \mathrm{Sign}(sk, m)$: The signing algorithm takes $sk = (q, \mathbb{G}, G, \mathrm{H}, x)$ and a message m as input. It generates $r \leftarrow \mathbb{Z}_q$ and computes $R = rG$. Then, it sets $c = \mathrm{H}(R, xG, m)$ and $s = (r + cx) \bmod q$. Finally, it outputs a signature $\sigma := (R, s)$. (4) $b \leftarrow \mathrm{Verify}(vk, m, \sigma)$: The verification algorithm takes $vk = (q, \mathbb{G}, G, \mathrm{H}, X)$, $\sigma = (R, s)$, and a message m as input. It computes $c = \mathrm{H}(R, X, m)$, and outputs $b = 1$ if $sG = R + cX$ holds, and $b = 0$ otherwise. Although we omit the detail, the Schnorr signature scheme satisfies the correctness and sEUF-CMA in the algebraic group model and the random oracle model [37].

2.4 Blind Signatures

A (predicate) blind signature scheme PBS [10, 15] is an interactive scheme whereby a signer signs messages requested by a user. While the signer can verify that these messages satisfy some conditions, such as predicates, he/she cannot obtain any further information about the messages. PBS is defined with a family of efficiently computable predicates. Predicates can be verified by a predicate compiler P that runs in polynomial time to check whether messages satisfy the predicates. P takes a predicate $prd \in \{0,1\}^*$ and a message $m \in \{0,1\}^*$ as input and then outputs 1 if m satisfies prd and 0 otherwise. We recall the definition of PBS by Fuchsbauer et al. [15] below: (1) $par \leftarrow \mathrm{Setup}(1^\lambda)$: the setup algorithm takes a security parameter as input and then outputs a public parameter par; (2)

[1] https://www.getmonero.org/

$(sk, vk) \leftarrow \mathrm{KeyGen}(par)$: the key generation algorithm takes a public parameter par as input and then outputs a signing key sk and a verification key vk; (3) $(b, \sigma) \leftarrow \langle \mathrm{Sign}(sk, prd), \mathrm{User}(vk, prd, m) \rangle$: the signing algorithm is an interactive process, whereby the signer-side process takes a signing key and possibly a predicate prd as input and outputs a signature σ or an error $\perp$ while the user-side process takes a verification key vk, a message m, and possibly a predicate prd as input and outputs $b = 1$ or $b = 0$; (4) $b \leftarrow \mathrm{Verify}(vk, m, \sigma)$: the verification algorithm takes a verification key vk, a message m, and a signature σ as input and outputs $b = 1$ or $b = 0$.

We note that the predicate blind Schnorr signature scheme PBSch by Fuchsbauer et al. [15] satisfies blindness and strong unforgeability for PBS although we omit the detail due to the page limitation.

3 Blind Adaptor Signatures

In this section, we define the syntax and security of (predicate) blind adaptor signatures. These definitions are our first contribution.

3.1 Syntax

A (predicate) blind adaptor signature (P)BAS is defined with a standard signature scheme $\mathrm{Sig} = (\mathrm{Setup}, \mathrm{KeyGen}, \mathrm{Sign}, \mathrm{Verify})$ and a relation R, which is common with ASes. Hereafter, we refer to these algorithms as (P)BAS.Setup, (P)BAS.KeyGen, (P)BAS.Sign, and (P)BAS.Verify. Then, a (P)BAS consists of the following algorithms, where we also inclde $(par) \leftarrow \mathrm{(P)BAS.Setup}(1^\lambda)$, and $(sk, vk) \leftarrow \mathrm{(P)BAS.KeyGen}(par)$ in the syntax for $(Y, y) \in \mathrm{R}$.

$(b, \hat{\sigma}) \leftarrow \langle \mathrm{pSign}(sk, prd, Y), \mathrm{User}(vk, prd, m, Y) \rangle$: The pre-signing algorithm is an interactive protocol between a signer and a user. The signer-side process takes a signing key sk, possibly a predicate prd, and a statement Y as input, while the user-side process takes a verification key vk, possibly a predicate prd, a message m, and a witness y as input, respectively. Then, the signer-side process and the user-side process interact with each other N_p times to generate a pre-signature, as illustrated in Figure 2.
The signer-side process outputs $b = 1$ or $b = 0$, and the user-side process outputs a pre-signature $\hat{\sigma}$ at the N_p-th time or an error $\perp$.

$b \leftarrow \mathrm{pVerify}(vk, m, \hat{\sigma}, Y)$: The pre-verification algorithm takes a verification key vk, a message m, a pre-signature $\hat{\sigma}$, and a statement Y as input, and then outputs $b = 1$ or $b = 0$.

$\sigma \leftarrow \mathrm{Adapt}(vk, \hat{\sigma}, y)$: The adaptation algorithm takes a verification key vk, a pre-signature $\hat{\sigma}$, and a witness y as input, and then outputs a signature σ.

$y \leftarrow \mathrm{Ext}(\sigma, \hat{\sigma}, Y)$: The extraction algorithm takes a signature σ, a pre-signature $\hat{\sigma}$, and a statement Y as input and then outputs a witness y.

The overview of the above syntax is illustrated in Fig. 2. A (P)BAS scheme consists of the following steps: A user first generates a pair $(y, Y) \in \mathrm{R}$ where y is a witness and Y is a statement satisfying an NP relation R, while the signer generates a pair of secret and verification keys through KeyGen. Subsequently, the signer and user engage in an interactive protocol $\langle \mathrm{pSign}(sk, prd, Y), \mathrm{User}(vk, prd, m, Y) \rangle$ to generate a pre-signature $\hat{\sigma}$. After they update their intermediate message and state, i.e., $msg_{U,i}$ and $state_{U,i}$ for the user and $msg_{S,i}$ and $state_{S,i}$ for the signer as the i-th output, a pre-signature $\hat{\sigma}$ is output from the user-side, not the signer-side. The user then executes Adapt to generate a signature σ from $\hat{\sigma}$ and stores it into blockchains.

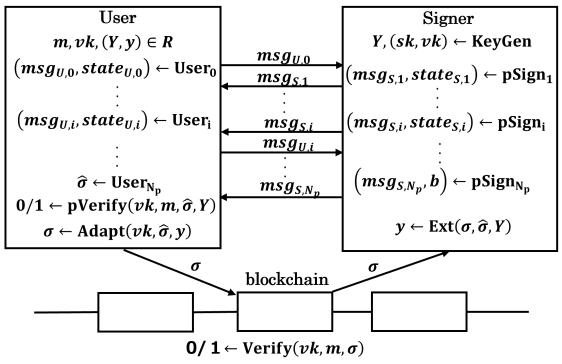

Figure 2: Overview of the blind adaptor signature protocol between a user and a signer.

Finally, the signer retrieves σ from the blockchain and extracts y from $\hat{\sigma}$ and σ through Extract.

The above syntax has two differences from that of Qin et al. [11]. First, our syntax captures interactions between a signer and a user in the pre-signing algorithm. It allows an adversary to utilize more interfaces during the interaction between pSign and User, and hence, we can discuss stronger security definitions. Another difference is that the above syntax potentially includes a predicate prd. It is also identical to the definition of standard BSes by removing prd. This means that our syntax is more general than that of Qin et al. because it includes predicates. We denote the above syntax by a PBAS scheme when a predicate prd is strictly included in the above syntax.

3.2 Security Definitions

We define the security of a (P)BAS scheme. We describe correctness, full extractability, unlinkability and pre-verify soundness, and describe the remaining security definitions in Appendix D. The following definitions are for a PBAS scheme that generates pre-signatures through an N_p-round interaction between the signer and the user, while they contain definitions without predicates by removing prd and replacing sequences with sets for each query. We also denote the full extractability of a (P)BAS scheme by bFExt in order to distinguish the existing full extractability by Dai et al. of non-blind ASes for the sake of convenience. We denote by $[n]$ a positive integer for a set $\{1, ..., n\}$. We denote an empty sequence by $[\,]$, the addition of a value a to a sequence $\vec{V}$ by $\vec{V} = \vec{V} \,||\, a$, the i-th element of $\vec{v}$ by $\vec{V}_i$, the size of $\vec{V}$ by $|\vec{V}|$, an empty set by $\emptyset$ and the addition of a set $\{a\}$ to a set S by $S = S \cup \{a\}$.

Definition 3.1 (Correctness). We say that a BAS scheme PBAS satisfies correctness if, the probability $\Pr[\mathrm{Correct}^m_{\mathrm{PBAS}}(\lambda) = 1] = 1$ holds for any security parameter λ, message m, and relation R, where $\mathrm{Correct}^m_{\mathrm{PBAS}}$ is the game shown in Fig. 3 and the pre-signing algorithm is with N_p interactions for some $N_p \in \mathbb{N}$.

The reason for the condition $b_1 \wedge b_2 \wedge b_3$ at Line 8 in Fig. 3 is that both pre-signatures and signatures should return 1 in their verification algorithms as well as (Y, y) should satisfy the given relation R.

Definition 3.2 (Full Extractability). We say that a PBAS scheme satisfies full extractability if, for any probabilistic polynomial time

1: $\mathrm{Correct}^{m}_{\mathrm{PBAS}}(\lambda)$
2: $par \leftarrow \mathrm{PBAS.Setup}(1^\lambda);\ (sk, vk) \leftarrow \mathrm{PBAS.KeyGen}(par)$
3: $(Y, y) \leftarrow \mathrm{R.Gen}(1^\lambda)$
4: $(\cdot, \hat{\sigma}) \leftarrow \langle \mathrm{PBAS.pSign}(sk, prd, Y), \mathrm{PBAS.User}(vk, prd, m, Y)\rangle$
5: $b_1 \leftarrow \mathrm{PBAS.pVerify}(vk, m, \hat{\sigma}, Y);\ \sigma \leftarrow \mathrm{PBAS.Adapt}(vk, \hat{\sigma}, y)$
6: $b_2 \leftarrow \mathrm{PBAS.Verify}(vk, m, \sigma);\ y \leftarrow \mathrm{PBAS.Ext}(\sigma, \hat{\sigma}, Y)$
7: $b_3 \leftarrow (Y, y) \in \mathrm{R}$
8: **return** $b_1 \wedge b_2 \wedge b_3$

Figure 3: The correctness game $\mathrm{Correct}^{m}_{\mathrm{PBAS}}(\lambda)$ **for** PBAS

adversary $\mathcal{A}$, $\mathrm{Adv}^{\mathrm{bFExt}}_{\mathrm{PBAS},\mathcal{A}}(\lambda) = \Pr[\mathrm{bFExt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda) = 1]$ that $\mathcal{A}$ wins the game shown in Fig. 4 is negligible for a security parameter λ.

1: $\mathrm{bFExt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda)$
2: $par \leftarrow \mathrm{PBAS.Setup}(1^\lambda);\ (sk, vk) \leftarrow \mathrm{PBAS.KeyGen}(par)$
3: $T[m] := \emptyset$ // m is an arbitrary message
4: $\vec{L} := [\,];\ \vec{PRD} := [\,];\ \vec{W} := [\,]$
5: $(m^*_i, \sigma^*_i, \{(Y_{i,j}, \hat{\sigma}_{i,j})\})_{i\in[n], j\in[n']} \leftarrow \mathcal{A}^{\mathrm{NewY,Sign,pSign}}(vk)$
6: $T[m^*_i] = T[m^*_i] \cup \{(Y_{i,j}, \hat{\sigma}_{i,j})\}$ // $\forall i \in [n]\ \forall j \in [n']$
7: **if** $(n > 0$
8: $\ \wedge \forall i \in [n] : \mathrm{PBAS.Verify}(vk, m^*_i, \sigma^*_i) = 1$
9: $\ \wedge \forall i \neq j \in [n] : (m^*_i, \sigma^*_i) \neq (m^*_j, \sigma^*_j)$
10: $\ \wedge \nexists f \in \mathrm{InjF}([n], [|\vec{PRD}|]) : \forall i \in [n] : \mathrm{P}(\vec{PRD}_{f(i)}, m^*_i) = 1$
11: $\ \wedge \exists i \in [n]\ \exists j \in [n'] : \forall (Y_{i,j}, \hat{\sigma}_{i,j}) \in T[m^*_i]\ st\ Y_{i,j} \notin \vec{L} :$
12: $\quad (Y_{i,j}, \mathrm{PBAS.Ext}(\sigma^*_i, \hat{\sigma}_{i,j}, Y_{i,j})) \notin \mathrm{R})$ **then return** 1
13: **else return** 0
14: NewY()
15: $(Y, \cdot) \leftarrow \mathrm{R.Gen}(1^\lambda);\ \vec{L} = \vec{L} \,||\, Y$; **return** Y
16: Sign(m)
17: $\sigma \leftarrow \mathrm{PBAS.Sign}(sk, m)$; **return** σ
18: pSign(prd, msg, Y, i, j)
19: **if** $\vec{W}_i = \perp$ **then return** $\perp$
20: **if** $j = 1$ **then**
21: $\quad (msg', state) \leftarrow \mathrm{PBAS.pSign}_1(sk, prd, msg, Y)$
22: $\quad \vec{W}_i = \vec{W}_i \,||\, (state, prd)$; **return** msg'
23: **if** $j \in [2, N_p - 1]$ **then**
24: $\quad (state, prd) := \vec{W}_i$
25: $\quad (msg', state) \leftarrow \mathrm{PBAS.pSign}_j(state, msg)$
26: $\quad \vec{W}_i = \vec{W}_i \,||\, (state, prd)$; **return** msg'
27: **if** $j = N_p$ **then**
28: $\quad (state, prd) := \vec{W}_i;\ (msg', b) \leftarrow \mathrm{PBAS.pSign}_{N_p}(state, msg)$
29: $\quad$ **if** $b = 1$ **then**
30: $\quad\quad \vec{W}_i = \perp;\ \vec{PRD} = \vec{PRD} \,||\, prd$; **return** msg'
31: **return** $\perp$

Figure 4: The full extractability game $\mathrm{bFExt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda)$ **for** PBAS**.**

In Fig.4, let $\mathrm{InjF}(B, D)$ be a set of all injective functions from a set B to a set D, and $\mathrm{P} : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}$ be a predicate compiler which takes a predicate prd and a message m as input and then outputs 1 if m satisfies prd and 0 otherwise.

The reasons for including n tuples, including multiple pairs of statements and pre-signatures for each message, in a forgery by $\mathcal{A}$ and the conditions from Line 7 to Line 12 are because of a combination of BSes and ASes. The strong unforgeability of (P)BSes [15] discusses n pairs of messages and signatures after n signing sessions in parallel at Line 5. If a user outputs a forgery of signatures, there exist successfully closed signing sessions among the n pairs. It also means that, for the number $|\vec{PRD}|$ of the closed signing sessions , there exists an injective mapping $f : [n] \to [|\vec{PRD}|]$ such that $\mathrm{P}(\vec{PRD}_{f(i)}, m^*_i) = 1$ holds. From the standpoints of the full extractability game, the adversary must output a tuple of message-signature pairs whereby no such an injective mapping f exists at Line 10. Meanwhile, the (full) extractability of ASes [13] guarantees the signer's security where the user cannot forge any pair of a message and a signature such that the verification returns 1 (at Line 8) while the witness cannot be extracted (at Line 11 and Line 12).

We note that the above full extractability is stronger than the one-more unforgeability and the witness extractability by Qin et al. [11]. Their security definitions allow an adversary to call only a single challenge query and a single pre-signature for each message, which follows Aumayr et al. [3]. In contrast, inspired by Dai et al. [13], the full extractability allows an adversary to obtain multiple pre-signatures on the same message and, consequently, is suitable for a wider range of practical applications. We also prove that our full extractability implies the one-more unforgeability and the witness extractability for BASes by inheriting from the original motivation of full extractability by Dai et al. [13]. (We prove this relationship in Section 3.3 and See Appendix D for the definitions of the one-more unforgeability and the witness extractability.)

Definition 3.3 (Unlinkability). We say that a PBAS scheme satisfies unlinkability if, for any probabilistic polynomial time adversary $\mathcal{A}$, the probability

$$\mathrm{Adv}^{\mathrm{Unlink}}_{\mathrm{PBAS},\mathcal{A}}(\lambda) = \left|\Pr[\mathrm{Unlink}^{\mathcal{A},1}_{\mathrm{PBAS}}(\lambda)] - \Pr[\mathrm{Unlink}^{\mathcal{A},0}_{\mathrm{PBAS}}(\lambda)]\right|$$

that $\mathcal{A}$ wins the game shown in Fig. 5 is negligible for a security parameter λ and $b \leftarrow \{0,1\}$.

The unlinkability ensures that adapted signatures are indistinguishable from standard ones [13], preserving privacy in trading. In contrast with Definition 3.2 of the full extractability, the SignChl oracle in the above definition is a natural extension of that in the original definition by Dai et al. [13] of unlinkability. Instead, the signing oracle for pre-signatures in our definition is different from the definition by Dai et al.: for instance, it is separated into User and pSign with interfaces for each step of the interaction. It means that adapted signatures are indistinguishable from standard signatures even if an adversary can obtain intermediate messages and states through rich attack interfaces.

The pre-verify soundness guarantees that, for any statement $Y \notin \mathbb{L}_{\mathrm{R}}$, there exists no pre-signature that cannot be transformed into valid signatures even with a witness. It guarantees the user-side security against a malicious signer [14].

Definition 3.4 (Pre-Verify Soundness). We say that a PBAS scheme satisfies pre-verify soundness if, for polynomially-bounded statement $Y \notin \mathbb{L}_{\mathrm{R}}$ and key $(sk, vk) \leftarrow \mathrm{PBAS.KeyGen}(par)$, $\Pr[\exists(m, \hat{\sigma}) :$

$\underline{\text{Unlink}^{\mathcal{A},b}_{\text{PBAS}}(\lambda)}$
$par \leftarrow \text{PBAS.Setup}(1^\lambda)$; $(sk, vk) \leftarrow \text{PBAS.KeyGen}(par)$
$\vec{WU} := [\,]$; $\vec{WS} := [\,]$
$b' \leftarrow \mathcal{A}^{\text{SignChl,Sign,User,pSign}}(vk)$
if $b = b'$ **then return** 1
else return 0
$\underline{\text{SignChl}(m, prd, (Y, y))}$
if $(Y, y) \notin R$ **then return** $\perp$
$\hat{\sigma} \leftarrow \langle \text{PBAS.pSign}(sk, prd, msg, Y), \text{PBAS.User}(vk, prd, m, y) \rangle$
if $\hat{\sigma} = \perp$ **then return** $\perp$
$\sigma_0 \leftarrow \text{PBAS.Adapt}(vk, \hat{\sigma}, y)$; $\sigma_1 \leftarrow \text{PBAS.Sign}(sk, m)$
return σ_b
$\underline{\text{Sign}(m)}$
$\sigma \leftarrow \text{PBAS.Sign}(sk, m)$; **return** σ
$\underline{\text{User}(prd, msg, i, j)}$
if $\vec{WU}_i = \perp$ **then return** $\perp$
if $j = 0$ **then**
  $(msg', state) \leftarrow \text{PBAS.User}_0(vk, prd, msg)$
  $\vec{WU}_i = \vec{WU}_i \,||\, (state, prd)$
  return msg'
if $j \in [1, N_p - 1]$ **then**
  $(state, prd) := \vec{WU}_i$
  $(msg', state) \leftarrow \text{PBAS.User}_j(state, msg)$
  $\vec{WU}_i = \vec{WU}_i \,||\, (state, prd)$
  return msg'
if $j = N_p$ **then**
  $(state, prd) := \vec{WU}_i$
  $(msg', state) \leftarrow \text{PBAS.User}_{N_p}(state, msg)$
  if $\hat{\sigma} \neq \perp$ **then**
    $\vec{WU}_i = \perp$; **return** $\hat{\sigma}$
return $\perp$
$\underline{\text{pSign}(prd, msg, Y, i, j)}$
if $\vec{WS}_i = \perp$ **then return** $\perp$
if $j = 1$ **then**
  $(msg', state) \leftarrow \text{PBAS.pSign}_1(sk, prd, msg, Y)$
  $\vec{WS}_i = \vec{WS}_i \,||\, (state, prd)$; **return** msg'
if $j \in [2, N_p - 1]$ **then**
  $(state, prd) := \vec{WS}_i$
  $(msg', state) \leftarrow \text{PBAS.pSign}_j(state, msg)$
  $\vec{WS}_i = \vec{WS}_i \,||\, (state, prd)$; **return** msg'
if $j = N_p$ **then**
  $(state, prd) := \vec{WS}_i$
  $(msg', b) \leftarrow \text{PBAS.pSign}_{N_p}(state, msg)$
  if $b = 1$ **then**
    $\vec{WS}_i = \perp$; **return** msg'
return $\perp$

Figure 5: The unlinkability game $\text{Unlink}^{\mathcal{A},b}_{\text{PBAS}}$ for PBAS.

$\text{PBAS.pVerify}(vk, m, \hat{\sigma}, Y) = 1] = 0$ holds for any security parameter λ, where $\mathbb{L}_R$ is a language for a relation R.

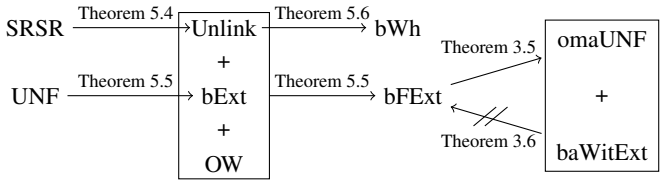

Figure 6: Relations among security notions

The above definition is a statistical definition, which is stronger than a game-based definition [14]. It guarantees that, for any statement $Y \notin \mathbb{L}_R$, a malicious signer is prevented from generating pre-signatures such that a user cannot obtain standard signatures even with valid witnesses. The pre-verify soundness is important for some kinds of protocols, such as oracle-based payment [20].

3.2.1 Security Gaps. There are security gaps between the existing schemes and our definitions. Although we describe the details in Appendix E due to the page limitation, we found that the scheme by Qin et al. [11] and the existing conversion [14] of the scheme by Fuchsbauer et al. [15] do not satisfy our definitions. We only mean that satisfying our definitions is non-trivial from the above schemes. We note that our definitions are out of the scope of the above schemes, and we do not mean that these schemes are insecure.

3.3 Relationships with Security Definitions

We demonstrate that the full extractability bFExt implies the one-more unforgeability omaUNF and witness extractability baWitExt defined in Appendix D but not vice versa for PBAS. The relations between the security definitions are summarized in Fig. 6, including the security proofs of our proposed schemes described later as Theorem 5.4, Theorem 5.5 and Theorem 5.6. Due to page limitations, we describe only the statements of Theorem 3.5 and Theorem 3.6, and show the one-more unforgeability and the witness extractability in Appendix D. We defer the proofs to the full version of the paper.

Theorem 3.5 (bFExt $\Rightarrow$ omaUNF + baWitExt). *Let* $\mathcal{A}_{\text{omaUNF}}$ *and* $\mathcal{A}_{\text{baWitExt}}$ *be adversaries against* omaUNF *and* baWitExt *games. Then there exist adversaries* $\mathcal{A}_{\text{bFExt},1}$ *and* $\mathcal{A}_{\text{bFExt},2}$ *against* bFExt *game such that the following equations hold:*

$$\Pr[\text{omaUNF}^{\mathcal{A}_{\text{omaUNF}}}_{\text{PBAS}}(\lambda) = 1] = \text{Adv}^{\text{bFExt}}_{\text{PBAS},\mathcal{A}_{\text{bFExt},1}}(\lambda), \tag{1}$$

$$\Pr[\text{baWitExt}^{\mathcal{A}_{\text{baWitExt}}}_{\text{PBAS}}(\lambda) = 1] = \text{Adv}^{\text{bFExt}}_{\text{PBAS},\mathcal{A}_{\text{bFExt},2}}(\lambda). \tag{2}$$

Theorem 3.6 (omaUNF + baWitExt $\not\Rightarrow$ bFExt). *Let* F *is a secure* PRF. *Then,* PBAS *without the full extractability* nfPBAS *defined in Fig.25 satisfies* omaUNF *and* baWitExt. *In other words, for any probabilistic polynomial time adversary* $\text{A}'_{\text{omaUNF}}$ *against the one-more unforgeability game and any probabilistic polynomial time adversary* $\text{A}'_{\text{baWitExt}}$ *against* baWitExt *for* nfPBAS*, we can construct adversaries* $\mathcal{A}_{\text{omaUNF}}$, $\mathcal{A}_{\text{baWitExt}}$, $\mathcal{A}_{\text{PRF},1}$ *and* $\mathcal{A}_{\text{PRF},2}$ *that satisfies the following inequalities hold:*

$$\begin{aligned}\Pr[\text{omaUNF}^{\mathcal{A}'_{\text{omaUNF}}}_{\text{nfPBAS}}(\lambda) = 1] \leq \Pr[\text{omaUNF}^{\mathcal{A}_{\text{omaUNF}}}_{\text{PBAS}}(\lambda) = 1] \\ + \text{Adv}^{\text{PRF}}_{\text{F},\mathcal{A}_{\text{PRF},1}}(\lambda),\end{aligned} \tag{3}$$

$$\Pr[\mathrm{baWitExt}^{\mathcal{A}'_{\mathrm{baWitExt}}}_{\mathrm{nfPBAS}}(\lambda)=1] \le \Pr[\mathrm{baWitExt}^{\mathcal{A}_{\mathrm{baWitExt}}}_{\mathrm{PBAS}}(\lambda)=1] + \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{F},\mathcal{A}_{\mathrm{PRF},2}}(\lambda). \quad (4)$$

However, nfPBAS *does not satisfy the full extractability. There exists an adversary* $\mathcal{A}_{\mathrm{bFExt}}$ *against the full extractability game of* nfPBAS. *The advantage of the adversary is* $\mathrm{Adv}^{\mathrm{bFExt}}_{\mathrm{nfPBAS},\mathcal{A}_{\mathrm{bFExt}}}(\lambda) = 1/2$.

Although the above theorems are for a PBAS scheme, we can also prove theorems for a BAS scheme by removing prd and replacing sequences with sets for each query in the games. Due to the page limitation, we omit these proofs and show them in our full paper.

4 Technical Overview

Our main idea is to leverage relations that support SRSR [13] with respect to statements and witnesses, and then we can utilize these relations instead of additional random numbers for blind signatures. We describe the above idea in detail below.

We first recall the security gap of the full extractability in the conversion of the scheme by Fuchsbauer et al. [15]. It is due to the additional random numbers generated by a user on the Schnorr signatures. When a user generates a pre-signature $\hat{\sigma}$, a user generates additional random numbers $(\alpha, \beta) \in \mathbb{Z}_q^2$ and incorporates them into the signature components (R', s') as $R' = R+\beta G+\alpha \cdot vk$ and $s' = s+\beta$. While these random numbers support blindness by breaking algebraic structures, they also prevent a signer from obtaining the underlying signature (R, s) since he/she lacks the knowledge of (α, β).

In the proposed scheme PBASch, we focus on the fact that the above additional random number β and βG are in a discrete logarithm relation which is a SRSR relation, and β with witness y and βG with statement Y, while another random number α is utilized in the same manner Specifically, PBASch is designed by which a signer adds a statement $Y \in \mathbb{G}$ to the random component $R \in \mathbb{G}$ of the Schnorr signatures, and then let the new random component be $R' = R + Y + \alpha X$, where X is a public key. Likewise, the user also adds its witness y corresponding to the message component $s \in \mathbb{Z}_p$ of the Schnorr signatures $s' = s + y$ through the Adapt algorithm.

More precisely, a user sends a signer only a ciphertext C of a message encrypted with a public key encryption scheme PKE in the first round, and then sends a proof pi of a non-interactive zero-knowledge argument systemNArg together with the above re-randomization with relations. Such a construction reveals no other information with respect to the message (as we define these schemes in Appendix C), and hence satisfies the weak blindness. The signature components of PBASch are then identical to the Schnorr signatures, i.e., satisfying the unlinkability. The above construction is possible when SRSR is supported, because $(\hat{R}, s)$ is indistinguishable by SRSR. When the signer extracts the above witness y, it computes $s' - s$ from σ and $\hat{\sigma}$, which satisfies the full extractability. It also supports a membership test to identify whether a statement Y belongs to the language $\mathbb{L}_{\mathrm{R}}$ of relation R in the pre-verification algorithm in order to satisfy the pre-verify soundness.

We note that the unlinkability is also useful for proving the full extractability. For instance, to simplify a security proof of the full extractability, we can reduce the full extractability into the (non-full) extractability in Fig. 22 in Appendix D via the unlinkability. It can simulate the Sign oracle of the full extractability game by utilizing the pSign and Adapt algorithm, which is our new proof technique. It also enable us to reduce the full extractability into the unforgeability of the Schnorr signatures via a typical Schnorr-based blind signatures [15].

5 Proposed Scheme

In this section, we propose the PBAS scheme PBASch. This construction is our main contribution.

5.1 The Construction

Let Sch be Schnorr signatures, PKE be a public key encryption scheme, NArg be a non-interactive zero-knowledge argument scheme, and R be a discrete-logarithm relation as described in Section 2. Moreover, $\mathrm{Memb}(G, Y)$ is an algorithm that can verify whether the statement Y is a member of the language $\mathbb{L}_{\mathrm{R}}$, returning 1 if it is a member and 0 otherwise. It can be instantiated as membership tests using Koshelev's method [38]. For a positive integer n, we denote $[n]$ the set $\{1, ..., n\}$. The construction of PBASch is shown in Fig. 7, a signer and a user generate a pre-signature through the two-round interaction. Since the discrete-logarithm relation is an SRSR relation [13], the distribution of the adapted signature $\sigma = (\hat{R}, s')$ returned by PBASch.Adapt is computationally indistinguishable from the distribution of the standard signatures $\sigma = (R, s)$ generated by the Schnorr signature schemes. From the above observation, PBASch can satisfy the unlinkability, and it also implies that PBASch satisfies the full extractability under the unforgeability of the Schnorr signature schemes via the unlinkability, as well as the witness hiding. We are also able to show that PBASch satisfies the pre-verify soundness, the weak blindness and the adaptability.

5.2 Security of PBASch

We show that PBASch satisfies the security definitions in Section 3.2 and Appendix D below. In this section, we describe the theorems of PBASch and show only a proof of the unlinkability. The other proofs are described in Appendix F due to the page limitation.

Theorem 5.1 (Correctness). *Let* $\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}]$ *be a non-interactive zero-knowledge argument scheme that satisfies the correctness. Then, the* PBASch *scheme satisfies the correctness.*

Theorem 5.2 (Adaptability). *The predicate blind adaptor Schnorr signature* PBASch *scheme satisfies the adaptability.*

Theorem 5.3 (Weak Blindness). *Let* PKE *be a public key encryption scheme and* $\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}]$ *be a non-interactive zero-knowledge argument scheme for some relation* $\mathrm{R}_{\mathrm{Sch}}$. *Also, for any* $b \in \{0, 1\}$, *let* Z_b *be a probabilistic polynomial time adversary against the zero knowledge game defined in Fig. 16 for* $\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}]$ *and* C_b *be a probabilistic polynomial time adversary against the IND-CPA game defined in Fig. 15 for* PKE. *Then, for any probabilistic polynomial time adversary* $\mathcal{A}$ *against* PBASch *in the weak blindness game defined in Fig. 20, the following inequality holds:*

$$\mathrm{Adv}^{\mathrm{wBLD}}_{\mathrm{PBASch},\mathcal{A}}(\lambda) \le \mathrm{Adv}^{\mathrm{ZK}}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}],\mathrm{Z}_0}(\lambda) + \mathrm{Adv}^{\mathrm{ZK}}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}],\mathrm{Z}_1}(\lambda) + \mathrm{Adv}^{\mathrm{CPA}}_{\mathrm{PKE},\mathrm{C}_0}(\lambda) + \mathrm{Adv}^{\mathrm{CPA}}_{\mathrm{PKE},\mathrm{C}_1}(\lambda). \quad (5)$$

Theorem 5.4 (Unlinkability). *Let* R *be some relation. Assume that there exists a probabilistic polynomial time adversary* S_b *in the SRSR game in Fig.1 for the discrete logarithm hardness* DL. *Then, for any probabilistic polynomial time adversary* $\mathcal{A}$ *against* PBASch

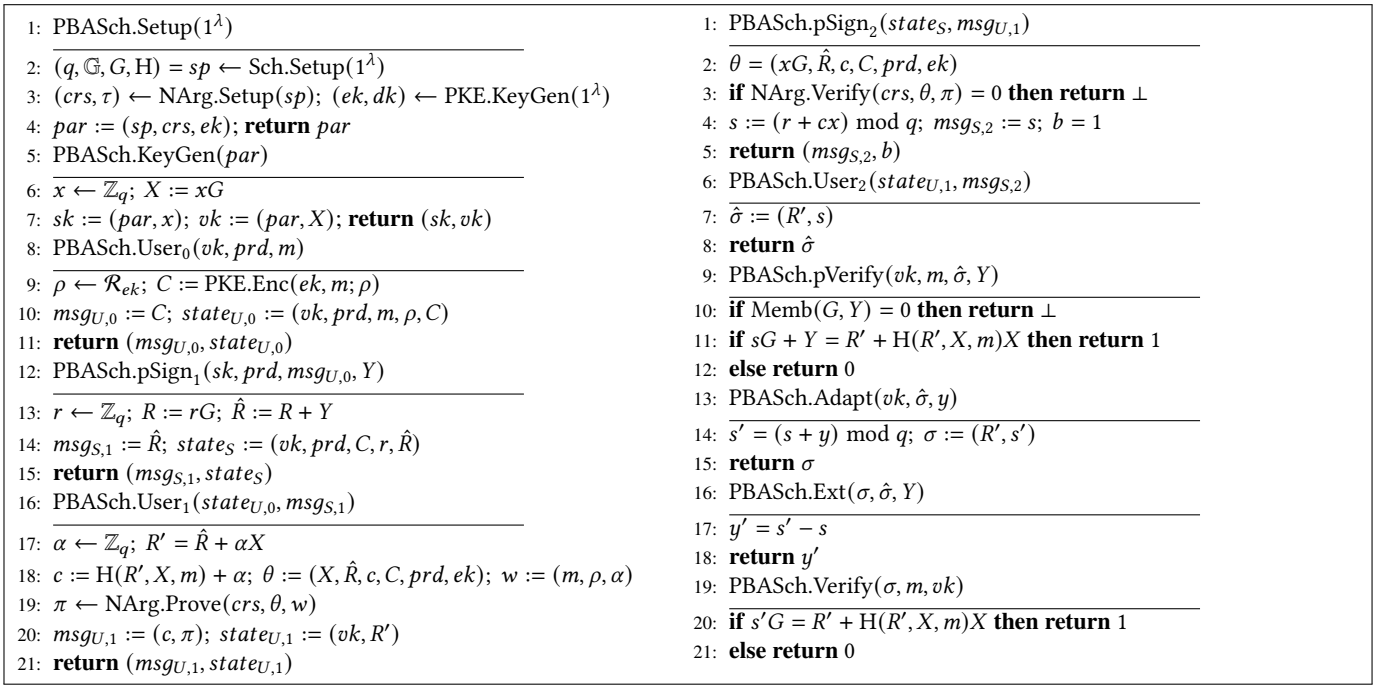

Figure 7: The predicate blind adaptor Schnorr signature PBASch **scheme**

in the unlinkability game in Fig.7, the following inequality holds:

$$\mathrm{Adv}^{\mathrm{Unlink}}_{\mathrm{PBASch},\mathcal{A}}(\lambda) \le \mathrm{Adv}^{\mathrm{SRSR}}_{\mathrm{DL},\mathrm{S}_0}(\lambda) + \mathrm{Adv}^{\mathrm{SRSR}}_{\mathrm{DL},\mathrm{S}_1}(\lambda). \quad (6)$$

Since the relation R *is the discrete-logarithm relation, and it is the SRSR relation [13],* PBASch *satisfies the unlinkability.*

Theorem 5.5 (Full Extractability). *Let $\mathcal{A}'$ be a probabilistic polynomial time adversary against* PBASch *in the unlinkability game,* F *be a probalistic polynomial time adversary against the sEUF-CMA game for* Sch, N *be a probabilistic polynomial time adversary against the soundness game for* NArg[$\mathrm{R}_{\mathrm{Sch}}$], D *be a probabilistic polynomial time adversary against the game* DL *and* O *be a probabilistic polynomial time adversary in the q-one-wayness game for the discrete-logarithm relation. Then, for any adversary $\mathcal{A}$ against* PBASch *in the full extractability game in Fig.7, the following inequality holds:*

$$\begin{aligned}\mathrm{Adv}^{\mathrm{bFExt}}_{\mathrm{PBASch},\mathcal{A}}(\lambda) \le\ & \mathrm{Adv}^{\mathrm{Unlink}}_{\mathrm{PBASch},\mathcal{A}'}(\lambda) + \mathrm{Adv}^{\mathrm{sEUF-CMA}}_{\mathrm{Sch,F}}(\lambda) \\ & + \mathrm{Adv}^{\mathrm{SND}}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}],\mathrm{N}}(\lambda) + q \cdot \exp(1) \cdot \mathrm{Adv}^{\mathrm{DL}}_{\mathrm{GrGen,D}}(\lambda) \\ & + \mathrm{Adv}^{\mathrm{q-OW}}_{\mathrm{DL,O}}(\lambda). \end{aligned} \quad (7)$$

Proof of Theorem 5.5 : We show that PBASch satisfies the full extractability by reducing the security properties to its building blocks. We proceed with game transformations shown in Fig .8. The games are as follows:

G_0 : In Fig.8, G_0 is the game where all colored boxes are ignored. The game is the full extractability game in Fig. 4 for PBASch. The information $S[m]$ and U held by the reduction algorithm at Line 5 differs from the game in Fig. 4.

G_1 : G_1 is obtained by adding the processing in the light green box to G_0. This game generates adapted signatures σ by using PBASch.pSign, PBASch.User and PBASch.Adapt instead of PBASch.Sign.

G_2 : G_2 is obtained by adding the processing in the orange box to G_1. In addition to verifying that the witness cannot be extracted from the signatures in $T[m^*_i]$, we also require that the witness cannot be extracted from the signatures in $S[m^*_i]$.

G_3, G_4 : G_3 is obtained by adding the process of the blue box to G_2, while G_4 is obtained by adding the process of the yellow box to G_2. We divide the conditions of G_2 into those of G_3 and G_4. These two games are the last games, which are parallel cases in this proof. In G_3, the adversary wins if it cannot extract a witness for any statement in the sequence $\vec{L}$, and is reduced to the strong unforgeability of the (non-adaptor) BS scheme PBSch [15] through the (non-full) extractability as described in Section 5.2. On the other hand, in G_4, the adversary wins if it can extract a witness for at least one statement, and is reduced to the q-one-wayness.

G_0 and G_1. The difference between G_0 and G_1 is whether the Sign oracle returns a standard signature or an adapted signature. Due to Theorem 5.4, since PBASch satisfies the unlinkability, any adversary cannot distinguish the difference in the output of the Sign oracle.

$$\Pr[\mathrm{G}_0] \le \Pr[\mathrm{G}_1] + \mathrm{Adv}^{\mathrm{Unlink}}_{\mathrm{PBASch},\mathcal{A}} \quad (8)$$

G_1 and G_2. In G_2, since $\forall i \in [n] : (m^*_i, \sigma^*_i) \notin U$, $S[m^*_i]$ is the empty set for the forgery message m^*_i. Therefore, b_2 is always true, and thus we have the following equation:

$$\Pr[\mathrm{G}_1] = \Pr[\mathrm{G}_2] \quad (9)$$

G_2, G_3 and G_4. We separate the winning condition of G_2 into

$\mathrm{bFExt}^{\mathcal{A}}_{\mathrm{PBASch}}(\lambda)$, $\mathrm{G}^{\mathcal{A}}_0$, $\mathrm{G}^{\mathcal{A}}_1$, $\mathrm{G}^{\mathcal{A}}_2$, $\mathrm{G}^{\mathcal{A}}_3$, $\mathrm{G}^{\mathcal{A}}_4$

$(q, \mathbb{G}, G, \mathrm{H}) = sp \leftarrow \mathrm{Sch.Setup}(1^\lambda)$
$(crs, \tau) \leftarrow \mathrm{NArg.Setup}(sp)$; $(ek, dk) \leftarrow \mathrm{PKE.KeyGen}(1^\lambda)$
$(sk, vk) \leftarrow \mathrm{PBASch.KeyGen}(sp)$
$T[m] := \emptyset$; $S[m] := \emptyset$; $U = \emptyset$ // m is an arbitrary message
$\vec{L} := [\,]$; $\vec{W} := [\,]$; $P\vec{R}D := [\,]$
$(m^*_i, \sigma^*_i, \{(Y_{i,j}, \hat{\sigma}_{i,j})\})_{i\in[n],j\in[n']} \leftarrow \mathcal{A}^{\mathrm{NewY,Sign,pSign}}(vk)$
if $\exists i \in [n] : (m^*_i, \sigma^*_i) \in U$ **then return** $\bot$
$T[m^*_i] = T[m^*_i] \cup \{(Y_{i,j}, \hat{\sigma}_{i,j})\}$ // $\forall i \in [n]\ \forall j \in [n']$
$b_1 = (n > 0$
  $\wedge \forall i \in [n] : \mathrm{Sch.Verify}(vk, m^*_i, \sigma^*_i) = 1$
  $\wedge \forall i \neq j \in [n] : (m^*_i, \sigma^*_i) \neq (m^*_j, \sigma^*_j)$
  $\wedge \nexists f \in \mathrm{InjF}([n], [|P\vec{R}D|]) : \forall i \in [n] : \mathrm{P}(P\vec{R}D_{f(i)}, m^*_i) = 1)$
  $\wedge \exists i \in [n]\ \exists j \in [n'] : \forall (Y_{i,j}, \hat{\sigma}_{i,j}) \in T[m^*_i]\ st\ Y_{i,j} \notin \vec{L}$:
    $(Y_{i,j}, \mathrm{PBASch.Ext}(\sigma^*_i, \hat{\sigma}, Y_{i,j})) \notin \mathrm{R})$
$b_2 = (\exists i \in [n] : \forall (Y, \hat{\sigma}) \in S[m^*_i]\ st\ Y \notin \vec{L}$:
    $\mathrm{PBASch.Ext}(\sigma^*_i, \hat{\sigma}, Y)) \notin \mathrm{R})$
$b_3 = (\exists i \in [n]$:
    $\forall (Y, \hat{\sigma}) \in T[m^*_i] \cup S[m^*_i]\ st\ Y \in \vec{L}$:
    $(Y, \mathrm{PBASch.Ext}(\sigma^*_i, \hat{\sigma}, Y)) \notin \mathrm{R})$
return b_1
return $b_1 \wedge b_2$
return $b_1 \wedge b_2 \wedge b_3$
return $b_1 \wedge b_2 \wedge \neg b_3$
NewY()
$(Y, \cdot) \leftarrow \mathrm{R.Gen}(1^\lambda)$; $\vec{L} = \vec{L} \,||\, Y$
return Y
Sign(m) // G_0
$\sigma \leftarrow \mathrm{PBASch.Sign}(sk, m)$
$U = U \cup \{(m, \sigma)\}$; **return** σ
Sign(m) // $\mathrm{G}^{\mathcal{A},b}_1$, $\mathrm{G}^{\mathcal{A},b}_2$, $\mathrm{G}^{\mathcal{A},b}_3$, $\mathrm{G}^{\mathcal{A},b}_4$
$(Y, y) \leftarrow \mathrm{R.Gen}(1^\lambda)$
$\hat{\sigma} \leftarrow \langle \mathrm{PBASch.pSign}(sk, \cdot, Y), \mathrm{PBASch.User}(vk, \cdot, m, y)\rangle$
$S[m] = S[m] \cup \{(Y, \hat{\sigma})\}$
$\sigma \leftarrow \mathrm{PBASch.Adapt}(vk, \hat{\sigma}, y)$
$U = U \cup \{(m, \sigma)\}$; **return** σ
pSign(prd, msg, Y, i, j)
if $\vec{W}_i = \bot$ **then return** $\bot$
if $j = 1$ **then**
  $C := msg$
  $r \leftarrow \mathbb{Z}_q$; $R = rG$; $\hat{R} = R + Y$
  $\vec{W}_i = \vec{W}_i \,||\, ((sk, prd, C, r, \hat{R}), prd)$
  return $\hat{R}$
if $j = 2$ **then**
  $((sk, prd, C, r, R'), prd) := \vec{W}_i$; $\theta := (xG, R', c, C, prd, ek)$
  if $\mathrm{NArg.Verify}(crs, \theta, \pi) = 0$ **then return** $\bot$
  $s = r + cx$; $\hat{\sigma} := (R', s)$
  $\vec{W}_i = \bot$; $P\vec{R}D = P\vec{R}D \,||\, prd$
  return s
return $\bot$

Figure 8: The full extractability game for PBASch**.**

G_3 and G_4. Then, the following condition holds in general:

$$\Pr[\mathrm{G}_2] = \Pr[\mathrm{G}_3] + \Pr[\mathrm{G}_4]. \tag{10}$$

Reduction of G_3 to the Extractability. Consider a probabilistic polynomial time adversary E against the extractability game, as shown in Fig.9. E simulates the NewY oracle and the signing oracle SignSim by using PBASch.pSign and PBASch.User algorithms. When the adversary $\mathcal{A}_{\mathrm{G}_3}$ wins G_3, E correspondingly wins the extractability game.

$$\Pr[\mathrm{G}_3] \leq \mathrm{Adv}^{\mathrm{bExt}}_{\mathrm{PBASch,E}}(\lambda) \tag{11}$$

$\mathrm{E}^{\mathrm{pSign}}(\lambda)$
$S[m] := \emptyset$; $U := \emptyset$
$\vec{L} := [\,]$
$(m^*_i, \sigma^*_i)_{i\in[n]} \leftarrow \mathcal{A}^{\mathrm{NewY,Sign,pSign}}_{\mathrm{G}_3}(vk)$
return $(m^*_i, \sigma^*_i)_{i\in[n]}$
NewY()
$(Y, \cdot) \leftarrow \mathrm{R.Gen}(1^\lambda)$; $\vec{L} = \vec{L} \,||\, Y$
return Y
SignSim(m)
$(Y, y) \leftarrow \mathrm{R.Gen}(1^\lambda)$
$\hat{\sigma} \leftarrow \langle \mathrm{PBASch.pSign}(sk, \cdot, Y), \mathrm{PBASch.User}(vk, \cdot, m, y)\rangle$
$S[m] = S[m] \cup \{(Y, \hat{\sigma})\}$
$\sigma \leftarrow \mathrm{PBASch.Adapt}(vk, \hat{\sigma}, y)$
$U = U \cup \{(m, \sigma)\}$
return σ

Figure 9: The adversary E **against the extractability game, using the adversary** $\mathcal{A}_{\mathrm{G}_3}$ **against the game** G_3**.**

Reduction of the Extractability to the Remaining Assumption. We describe a sketch of the remaining reduction from G_3 below due to the page limitation. We note that the pre-signing oracle pSign for PBASch in G_3 can be simulated by the signing oracle of the (non-adaptor) blind signature scheme by Fuchsbauer et al. [15], named PBSch for the sake of convenience. Indeed, the signature form of PBASch in G_3 is identical to PBSch because of the Schnorr-based constructions. We then construct the adversary U against the strong unforgeability game by utilizing the adversary $\mathcal{A}_{\mathrm{bExt}}$ against the extractability game for PBASch, which is shown in Fig.10.

$$\mathrm{Adv}^{\mathrm{bExt}}_{\mathrm{PBASch},\mathcal{A}}(\lambda) \leq \mathrm{Adv}^{\mathrm{UNF}}_{\mathrm{PBSch,U}}(\lambda), \tag{12}$$

where PBSch is the scheme by Fuchsbauer et al. [15]. Namely, PBSch transformed by G_3 is reduced to the strong unforgeability of the scheme by Fuchsbauer et al., and therefore, we can follow the same proof strategy of Theorem 1 in [15]. It leads to the following inequations, which can be reduced by using the same technique in [15]:

$$\begin{aligned}\mathrm{Adv}^{\mathrm{UNF}}_{\mathrm{PBSch},\mathcal{A}}(\lambda) \leq\ & \mathrm{Adv}^{\mathrm{sEUF-CMA}}_{\mathrm{Sch,F}}(\lambda) + \mathrm{Adv}^{\mathrm{SND}}_{\mathrm{NArg}[\mathrm{R_{Sch}}],\mathrm{N}}(\lambda) \\ & + q \cdot \exp(1) \cdot \mathrm{Adv}^{\mathrm{DL}}_{\mathrm{GrGen,D}}(\lambda).\end{aligned} \tag{13}$$

Reduction of G_4 to the q-One-Wayness. We construct the adversary O against the q-one-wayness game. This construction uses the adversary $\mathcal{A}_{\mathrm{G}_4}$ against the game G_4, as illustrated in Fig.11. $\mathcal{A}_{\mathrm{G}_4}$ winning the game G_4 implies successful extraction of a witness corresponding to a statement generated by the NewY oracle. This

$\mathrm{U}^{\mathrm{Sign}}(\lambda)$
$T[m] := \emptyset$
$\vec{W} := [\,]$; $P\vec{R}D := [\,]$
$(m_i^*, \sigma_i^*)_{i\in[n]} \leftarrow \mathcal{A}_{\mathrm{bExt}}^{\mathrm{pSign}}(vk)$
if $(n > 0$
$\wedge \forall i \in [n] : \mathrm{PBSch.Verify}(vk, m_i^*, \sigma_i^*) = 1$
$\wedge \forall i \neq j \in [n] : (m_i^*, \sigma_i^*) \neq (m_j^*, \sigma_j^*)$
$\wedge \nexists f \in \mathrm{InjF}([n], [|P\vec{R}D|]) : \forall i \in [n] : \mathrm{P}(P\vec{R}D_{f(i)}, m_i^*) = 1)$
then return $(m_i^*, \sigma_i^*)_{i\in[n]}$
$\mathrm{pSign}(prd, msg, Y, i, j)$
if $\vec{W}_i = \perp$ **then return** $\perp$
if $j = 1$ **then**
  $C := msg$
  $(R, state) \leftarrow \mathrm{PBSch.Sign}_1(sk, prd, C)$
  $\hat{R} = R + Y$
  $\vec{W}_i = ((sk, C, r, \hat{R}), prd)$
  return R'
if $j = 2$ **then**
  $c := msg$; $((sk, C, r, R'), prd) := \vec{W}_i$
  $(s, b) \leftarrow \mathrm{PBSch.Sign}_2(state, c)$
  if $b = 1$ **then**
    $\vec{W}_i = \perp$; $P\vec{R}D = P\vec{R}D \,||\, prd$
    return s
return $\perp$

Figure 10: The adversary U **against the strong unforgeability game, using the adversary** $\mathcal{A}_{\mathrm{Ext}}$ **against the extractability game for** PBASch

.

implies the extraction of a valid witness for at least one of the q given statements $Y_1, ..., Y_q$. Consequently, O wins the q-OW game. Therefore, we have the following equation:

$$\Pr[\mathrm{G}_4] \leq \mathrm{Adv}_{\mathrm{DL,O}}^{\mathrm{q-OW}}(\lambda). \tag{14}$$

Combining Equations (8) through (14), we bound the advantage of the full extractability game for PBASch as follows:

$$\mathrm{Adv}_{\mathrm{PBASch},\mathcal{A}}^{\mathrm{bFExt}}(\lambda) \leq \mathrm{Adv}_{\mathrm{PBASch},\mathcal{A}}^{\mathrm{Unlink}}(\lambda) + \mathrm{Adv}_{\mathrm{PBSch,U}}^{\mathrm{UNF}}(\lambda) + \mathrm{Adv}_{\mathrm{DL,O}}^{\mathrm{q-OW}}(\lambda).$$

This completes the proof of Theorem 5.5. □

Theorem 5.6 (Witness Hiding). *Let $\mathcal{A}'$ be a probabilistic polynomial time adversary against* PBASch *in the unlinkability game. Then, for any adversary $\mathcal{A}$ against* PBASch *in the witness hiding game in Fig.21, the following inequality holds:*

$$\mathrm{Adv}_{\mathrm{PBASch},\mathcal{A}}^{\mathrm{bWh}}(\lambda) \leq \mathrm{Adv}_{\mathrm{PBASch},\mathcal{A}'}^{\mathrm{Unlink}}(\lambda) \tag{15}$$

Theorem 5.7 (Pre-Verify Soundness). *If the algorithm* Memb *can check membership of the statement in the language $\mathbb{L}_{\mathrm{R}}$ efficiently, the* PBASch *scheme satisfies the pre-verify soundness.*

We show the proofs of Theorem 5.1, Theorem 5.2, Theorem 5.3, Theorem 5.4, Theorem 5.6, Theorem 5.7 in Appendix F due to the page limitation, and describe only their intuitions below. Since it is straightforward to check that the PBASch satisfies the correctness and adaptability, we omit the explanation of the proof for Theorem 5.1 and Theorem 5.2. In the security proof of Theorem 5.3, we first replace zero-knowledge proofs for a user with simulated proofs. Next, we replace a ciphertext of a user with that of a fixed message $\bar{m}$. Then, the outputs of the User_0 and User oracles no longer contain information about b. Thus, PBASch satisfies the weak blindness. The main idea of the proof for Theorem 5.4 is to reduce the unlinkability to SRSR by modifying the game in a manner that the SignChl oracle in Fig. 5 independently samples a statement Y' and witness y' inside the oracle itself. In these games, the distribution of adapted signatures is identical to that of standard signatures by virtue of SRSR. Theorem 5.6 is proven via the unlinkability game of PBASch. Here, a simulator, which generates standard signatures without a witness y, is replaced with the signing oracle of the standard

$\mathrm{O}(Y_1, ..., Y_q)$
$T[m] := \emptyset$; $S[m] := \emptyset$; $U := \emptyset$
$\vec{W} := [\,]$; $P\vec{R}D := [\,]$
$(m_i^*, \sigma_i^*, \{(Y_{i,j}, \hat{\sigma}_{i,j})\})_{i\in[n], j\in[n']} \leftarrow \mathcal{A}_{\mathrm{G_4}}^{\mathrm{NewY,Sign,pSign}}(vk)$
$T[m_i^*] = T[m_i^*] \cup \{(Y_{i,j}, \hat{\sigma}_{i,j})\}$ // $\forall i \in [n]\ j \in [n']$
if$(\exists i \in [n] : \mathrm{PBASch.Verify}(vk, m_i^*, \sigma_i^*) = 0)$ **then**
  return $\perp$
for $i \in [n] : (m_i^*, \sigma_i^*)$ **do**
  for $(j \in [n'] : (Y_{i,j}, \hat{\sigma}_{i,j}) \in T[m_i^*]\ st\ \exists I \in [q] : Y_{i,j} = Y_I)$ **do**
    $y \leftarrow \mathrm{PBASch.Ext}(\sigma_i^*, \hat{\sigma}_{i,j}, Y_{i,j})$
    if $(Y_{i,j}, y) \in \mathrm{R}$ **then return** (I, y)
return $\perp$
NewY()
$i := i + 1$; **return** Y_i
Sign(m)
$(Y, y) \leftarrow \mathrm{R.Gen}(1^\lambda)$
$(\cdot, \hat{\sigma}) \leftarrow \langle \mathrm{PBASch.pSign}(sk, \cdot, Y), \mathrm{PBASch.User}(vk, \cdot, m, y)\rangle$
$S[m] = S[m] \cup \{(Y, \hat{\sigma})\}$
$\sigma \leftarrow \mathrm{PBASch.Adapt}(vk, \hat{\sigma}, y)$
$U = U \cup \{(m, \sigma)\}$
return σ
$\mathrm{pSign}(prd, msg, Y, i, j)$
if $W[\ i\ ] = \perp$ **then return** $\perp$
if $j = 1$ **then**
  $C := msg$
  $r \leftarrow \mathbb{Z}_q$; $R = rG$
  $\hat{R} = R + Y$
  $\vec{W}_i = \vec{W}_i \,||\, ((sk, prd, C, r, \hat{R}), prd)$
  return $\hat{R}$
if $j = 2$ **then**
  $c := msg$
  $((sk, prd, C, r, \hat{R}), prd) := \vec{W}_i$
  $\theta := (xG, \hat{R}, c, C, prd, ek)$
  if $\mathrm{NArg.Verify}(crs, \theta, \pi) = 0$ **then return** $\perp$
  $s = r + cx$
  $\vec{W}_i = \perp$; $P\vec{R}D = P\vec{R}D \,||\, prd$
  return s
return $\perp$

Figure 11: The adversary O **against the q-one-wayness game, using the adversary $\mathcal{A}_{\mathrm{G_4}}$ against the game** $\mathrm{G_4}$

.

signatures. The oracles in the witness hiding game are simulated by the SignChl oracle in the unlinkability game. Theorem 5.7 is proven by the existence of Memb, which can efficiently verify whether a statement Y belongs to the language $\mathbb{L}_R$. Since pVerify incorporates this membership test, any statement $Y \notin \mathbb{L}_R$ will be rejected, ensuring the pre-verify soundness.

6 Application

In this section, we discuss privacy-enhancing applications that can be improved by PBASch. We discuss applications of the proposed scheme PBASch below. PBASch is suitable for applications that require both the conditional execution of transactions and their privacy-enhancing. It contributes to scenarios where users exchange cryptocurrency, where any predicate is satisfied without revealing their details. Specifically, we discuss privacy-enhancing oracle-based conditional payments and privacy-enhancing atomic swaps with fungibility below, although the detailed designs are in future work.

Privacy-Enhancing Pre-Scheduled Payments Pre-scheduled payments are a tool that allows a third-party to automatically perform payments based on a certain event in the real world, and a monthly subscription, such as Netflix, to provide a service is a typical scenario [20]. The pre-scheduled payments can be realized by oracle-based payments [20]: a sender, Alice, generates a pre-signature $\hat{\sigma}$ and encrypts a witness y. Then Alice sends both of them to a receiver, Bob, and he decrypts an witness y identical to payment information by receiving a signature σ from a third-party, called an oracle. It is considered that, for the above example of Netflix, Alice requests a coupon for using the service, and then she obtains a valid coupon with a BAS through interaction with Bob, i.e., Netflix as a service provider, while an oracle is a system that performs payments between them. In doing so, The proposed scheme PBASch can improve the privacy of Alice because Bob can sign the requested coupon without knowing the detailed information about Alice, such as her watch history and home address, by virtue of the weak blindness of plaintexts. Furthermore, Bob can always extract the witness after he receives a signature from the oracle by virtue of the full extractability. Additionally, the pre-verify soundness guarantees that every valid pre-signature can be adapted to its standard signature, thereby preventing adversaries from obtaining unauthorized coupons.

Improving Fungibility for Privacy-Enhancing Atomic Swaps Another application is to improve privacy-enhancing atomic swaps from a financial aspect. Atomic swaps enable both a sender, named Alice, and a receiver, named Bob, to exchange cryptocurrency in different blockchains for their trading. BlindHub [11] is a payment channel with BASes for atomic swaps, whereby Alice exchanges cryptocurrency with Bob via an untrusted intermediary, named Tumbler. However, since the scheme by Qin et al. [11] does not satisfy the unlinkability as described in Section 3.2.1, any third party can link whether the concrete coins on cryptocurrency are exchanged by atomic swaps [19]. Then, according to Hanzlik et al. [19], the fungibility of swapped coins is degraded because money should not be tainted by its origins: namely, all coins in currency have the same value independent of their history. The proposed scheme PBASch satisfied the unlinkability, and therefore, enables a payment channel to conceal whether coins are exchanged by atomic swaps, as well as providing the blindness of information on their trading. Namely, it can maintain the value of coins on cryptocurrency in addition to enhancing the privacy of atomic swaps.

7 Conclusion

In this paper, we proposed the first PBAS scheme that satisfies the unlinkability, the full extractability, and the pre-verify soundness. We also formalized the security of a (P)BAS scheme and proved a relationship between our security definitions and the existing security definitions, including security gaps from the existing schemes [11, 14]. Our idea to construct a PBAS scheme is to utilize relations that support random self-reducibility, and then utilize relations of adaptor signatures instead of additional random numbers for blind signatures. We proved the security of the proposed schemes and introduced a new proof technique for the full extractability via the unlinkability as well. PBASch can potentially improve not only the security in pre-scheduled payments [20] but also the fungibility of cryptocurrency in privacy-preserving atomic swap by virtue of the full extractability and the unlinkability. Future work includes the design of a construction with stronger witness hiding [21] and developing a generic construction of BASes based on our approach.

Acknowledgment. A part of this work was supported by JST PRESTO, Japan, Grant Number JPMJPR23P6.

References

[1] Andrew Poelstra. *Scriptless scripts*. Presentation Slides, https://download.wpsoftware.net/bitcoin/wizardry/mw-slides/2017-05-milan-meetup/slides.pdf. Accessed: 2024-08-22. 2017.

[2] Claus-Peter Schnorr. "Efficient signature generation by smart cards". In: *Journal of cryptology* 4 (1991), pp. 161–174.

[3] Aumayr et al. "Generalized channels from limited blockchain scripts and adaptor signatures". In: *Proc. of ASIACRYPT 2021*. Springer. 2021, pp. 635–664.

[4] Zijian Bao et al. "An Identity-based Adaptor Signature Scheme and its Applications in the Blockchain System". In: *IEEE Open Journal of the Computer Society* (2023).

[5] Yixing Zhu et al. "Adaptor signature based on randomized EdDSA in blockchain". In: *Digital Communications and Networks* (2024).

[6] Peifang Ni, Anqi Tian, and Jing Xu. "pipeSwap: Forcing the Early Release of a Secret for Atomic Swaps Across All Blockchains". In: *Cryptology ePrint Archive* (2024).

[7] Apoorvaa Deshpande and Maurice Herlihy. "Privacy-preserving cross-chain atomic swaps". In: *Proc. of FC 2020*. Vol. 12063. LNCS. Springer. 2020, pp. 540–549.

[8] Nurzhan Zhumabekuly Aitzhan and Davor Svetinovic. "Security and privacy in decentralized energy trading through multi-signatures, blockchain and anonymous messaging streams". In: *IEEE transactions on dependable and secure computing* 15.5 (2016), pp. 840–852.

[9] Qi Feng et al. "A survey on privacy protection in blockchain system". In: *Journal of network and computer applications* 126 (2019), pp. 45–58.

[10] David Chaum. "Blind signatures for untraceable payments". In: *Advances in Cryptology: Proc of Crypto 82*. Springer. 1983, pp. 199–203.

[11] Xianrui Qin et al. “BlindHub: Bitcoin-Compatible Privacy-Preserving Payment Channel Hubs Supporting Variable Amounts”. In: *IEEE S&P 2023*. IEEE, 2023, pp. 2462–2480.

[12] Xiaoming Hu and Haichan Chen. “Design and Analysis of an Anonymous and Fair Trading Scheme for Electronic Resources with Blind Adaptor Signature”. In: *Proc. of CISP-BMEI 2023*. IEEE. 2023, pp. 1–5.

[13] Wei Dai, Tatsuaki Okamoto, and Go Yamamoto. “Stronger security and generic constructions for adaptor signatures”. In: *International Conference on Cryptology in India*. Springer. 2022, pp. 52–77.

[14] Paul Gerhart et al. “Foundations of Adaptor Signatures”. In: *Proc. of EUROCRYPT 2024*. Springer, 2024, pp. 161–189.

[15] Georg Fuchsbauer and Mathias Wolf. “Concurrently Secure Blind Schnorr Signatures”. In: *Proc. of EUROCRYPT 2024*. Springer. 2024, pp. 124–160.

[16] Tim Ruffing Pieter Wuille Jonas Nick. *Schnorr Signatures for secp256k1*. https://bips.xyz/340. 2020.

[17] Gavin Wood. *Ethereum: A secure decentralised generalised transaction ledger Byzantium Version*. https://ethereum.github.io/yellowpaper/paper.pdf. 2022.

[18] Xiangyu Liu, Ioannis Tzannetos, and Vassilis Zikas. “Adaptor Signatures: New Security Definition and a Generic Construction for NP Relations”. In: *Proc. of ASIACRYPT 2024*. Vol. 15485. LNCS. Springer, 2025, pp. 168–193.

[19] Lucjan Hanzlik et al. “ Sweep-UC: Swapping Coins Privately ”. In: *Proc. of IEEE S&P 2024*. IEEE, 2024, pp. 3822–3839.

[20] Varun Madathil et al. “Cryptographic Oracle-based Conditional Payments”. In: *Proc. of NDSS 2023*. The Internet Society, 2023, pp. 1–18.

[21] Nikhil Vanjani, Pratik Soni, and Sri AravindaKrishnan Thyagarajan. “Functional Adaptor Signatures: Beyond All-or-Nothing Blockchain-based Payments”. In: *Proc. of CCS 2024*. ACM, 2024, pp. 1493–1507.

[22] Andreas Erwig, Faust, et al. “Two-party adaptor signatures from identification schemes”. In: *Proc. of PKC 2021*. Springer. 2021, pp. 451–480.

[23] Michele Ciampi et al. “Universal Adaptor Signatures from Blackbox Multi-party Computation”. In: *Proc. of CT-RSA 2025*. Ed. by Arpita Patra. Vol. 15598. LNCS. Springer, 2025, pp. 375–398.

[24] Masayuki Abe and Tatsuaki Okamoto. “Provably secure partially blind signatures”. In: *Proc. of CRYPTO 2000*. Springer. 2000, pp. 271–286.

[25] Noemi Glaeser et al. “Foundations of coin mixing services”. In: *Proc. of CCS 2022*. ACM, 2022, pp. 1259–1273.

[26] Elizabeth Crites et al. “Snowblind: A threshold blind signature in pairing-free groups”. In: *Proc. of CRYPTO 2023*. Springer. 2023, pp. 710–742.

[27] Erkan Tairi, Pedro Moreno-Sanchez, and Matteo Maffei. “Post-quantum adaptor signature for privacy-preserving off-chain payments”. In: *Proc. of FC 2021*. Springer. 2021, pp. 131–150.

[28] Binbin Tu, Min Zhang, and Chen Yu. “Efficient ECDSA-Based Adaptor Signature for Batched Atomic Swaps”. In: *Proc. of ISC 2022*. Springer. 2022, pp. 175–193.

[29] Ghous Amjad, Kevin Yeo, and Moti Yung. “Rsa blind signatures with public metadata”. In: *Cryptology ePrint Archive* (2023).

[30] Alex Davidson et al. “Privacy pass: Bypassing internet challenges anonymously”. In: *Proceedings on Privacy Enhancing Technologies* (2018).

[31] Stefano Tessaro and Chenzhi Zhu. “Short pairing-free blind signatures with exponential security”. In: *Proc. of EUROCRYPT 2022*. Springer. 2022, pp. 782–811.

[32] Rutchathon Chairattana-Apirom et al. “PI-cut-choo and friends: Compact blind signatures via parallel instance cut-and-choose and more”. In: *Proc. of CRYPTO 2022*. Springer. 2022, pp. 3–31.

[33] Lucjan Hanzlik, Julian Loss, and Benedikt Wagner. “Rai-choo! Evolving blind signatures to the next level”. In: *Proc. of EUROCRYPT 2023*. Springer. 2023, pp. 753–783.

[34] Zhimei Sui et al. “Monet: A fast payment channel network for scriptless cryptocurrency monero”. In: *Proc. of ICDCS 2022*. IEEE. 2022, pp. 280–290.

[35] Joseph K. Liu, Victor K. Wei, and Duncan S. Wong. “Linkable Spontaneous Anonymous Group Signature for Ad Hoc Groups”. In: *Proc. of ACISP 2004*. Berlin, Heidelberg: Springer, 2004, pp. 325–335.

[36] Shi-Feng Sun et al. “RingCT 2.0: A Compact Accumulator-Based (Linkable Ring Signature) Protocol for Blockchain Cryptocurrency Monero”. In: *Proc. of ESORICS 2017*. Vol. 10493. LNCS. Springer, 2017, pp. 456–474.

[37] Georg Fuchsbauer and Michele Orrù. “Non-interactive mimblewimble transactions, revisited”. In: *Proc. of ASIACRYPT 2022*. Vol. 13791. LNCS. Springer. 2022, pp. 713–744.

[38] Dmitrii Koshelev. “Correction to: Subgroup membership testing on elliptic curves via the Tate pairing”. In: *Journal of Cryptographic Engineering* 14.1 (2024), pp. 127–128.

[39] Yunfeng Ji et al. “Threshold/Multi Adaptor Signature and Their Applications in Blockchains”. In: *Electronics* 13.1 (2024).

[40] Zengxiang Wang et al. “TASS: Threshold Adaptor Signature Scheme for Holographic IoT Consumer Devices in Blockchain Enabled Federated Learning”. In: *IEEE Transactions on Consumer Electronics* (2025), pp. 1–1.

[41] Kaisei Kajita, Go Ohtake, and Tsuyoshi Takagi. “Consecutive Adaptor Signature Scheme: From Two-Party to N-Party Settings”. In: *Proc. of ProvSec 2024*. Vol. 14903. LNCS. Springer, 2025, pp. 23–42.

[42] Xinjie Zhu et al. “Two-Party Adaptor Signature Scheme Based on IEEE P1363 Identity-Based Signature”. In: *IEEE Open Journal of the Communications Society* 4 (2023), pp. 2717–2728.

A Further Related Works

We describe further related work on extensions of adaptor signatures, which take different directions from this paper. For advanced cryptography, to the best of our knowledge, there are the following extensions: threshold setting [39, 40] allows multiple signers to generate partial signatures and construct full signatures from them; multi-party setting [41] allows multiple signers to generate compact

single signatures; identity-based setting [42] allows each signer to utilize his/her own identifier as its public key; and functional setting [21] allows a user to embed functions into signatures and a signer to extract evaluation results of their witnesses. We believe that blindness can be introduced into Schnorr-based constructions [39, 41, 21] by combining the results of this paper.

B Figures of Preliminaries

In this section, we provide the figures associated with the preliminaries in the Section 2. These figures have been moved to the appendix to maintain the clarity and conciseness of the main body of the paper.

Fig. 12 shows the discrete-logarithm (DL) game of Definition 2.1.

$\mathrm{DL}^{\mathcal{A}}_{\mathrm{GrGen}}(\lambda)$
$(q, \mathbb{G}, G) \leftarrow \mathrm{GrGen}(1^\lambda)$
$x \leftarrow \mathbb{Z}_q$; $X := xG$
$x^* \leftarrow \mathcal{A}(q, \mathbb{G}, G, X)$
if $x^* = x$ **return** 1
else return 0

Figure 12: The DL game $\mathrm{DL}^{\mathcal{A}}_{\mathrm{GrGen}}(\lambda)$.

Fig. 13 illustrates the q-one-wayness (q-OW) game of Definition 2.3.

$\mathrm{qOW}^{\mathcal{A}}_{\mathrm{R}}(\lambda)$
For $i \in [q]$ do $(Y_i, \cdot) \leftarrow \mathrm{R.Gen}(1^\lambda)$
$(I, y') \leftarrow \mathcal{A}(Y_1, ..., Y_q)$ // $I \in [q]$
if $(Y_I, y') \in \mathrm{R}$
then return 1
else return 0

Figure 13: The q-OW game $\mathrm{qOW}^{\mathcal{A}}_{\mathrm{R}}(\lambda)$.

Fig. 14 illustrates the strongly existential unforgeability (sEUF-CMA) game of Definition 2.4.

$\mathrm{sEUF}^{\mathcal{A}}_{\mathrm{Sig}}(\lambda)$
$(sk, vk) \leftarrow \mathrm{Sig.KeyGen}(1^\lambda)$; $\vec{Q} := []$
$(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathrm{Sign}}(vk)$
if $\mathrm{Sig.Verify}(vk, m^*, \sigma^*) = 1 \wedge (m^*, \sigma^*) \notin \vec{Q}$ **then return** 1
else return 0
$\mathrm{Sign}(m)$
$\sigma \leftarrow \mathrm{Sig.Sign}(sk, m)$; $\vec{Q} = \vec{Q} \,||\, (m, \sigma)$; **return** σ

Figure 14: The sEUF-CMA game $\mathrm{sEUF}^{\mathcal{A}}_{\mathrm{Sig}}(\lambda)$.

C Additional Preliminaries

In this section, we define the building blocks of the proposed schemes.

C.1 Pseudorandom Function

Let λ be a security parameter. Let $\mathrm{RF}_\lambda := \{f \mid f : \{0,1\}^{l_1(\lambda)} \rightarrow \{0,1\}^{l_2(\lambda)}\}$ be a family of random functions. We say a function $\mathrm{F} : \{0,1\}^{l_1(\lambda)} \rightarrow \{0,1\}^{l_2(\lambda)}$ is a secure pseudorandom function PRF if, for any probabilistic polynomial time adversary $\mathcal{A}$, the probability $\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{F},\mathcal{A}}(\lambda)$ given by the following equation is negligible for λ.

$$\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{F},\mathcal{A}}(\lambda) = \left|\Pr[\mathcal{A}^{\mathrm{F}}(\lambda) = 1] - \Pr[\mathcal{A}^{f}(\lambda) = 1 \mid f \leftarrow \mathrm{RF}]\right|.$$

C.2 Public key Encryption

A public key encryption scheme PKE consists of the following algorithms: (1) $(ek, dk) \leftarrow \mathrm{KeyGen}(1^\lambda)$: The key generation algorithm takes a security parameter 1^λ as input and then outputs an encryption key ek and a decryption key dk, where ek defines a plaintext space $\mathcal{M}_{ek}$, a random number space $\mathcal{R}_{ek}$, and a ciphertext space $\mathcal{C}_{ek}$; (2) $C \leftarrow \mathrm{Enc}(ek, m; \rho)$: The encryption algorithm takes an encryption key ek and a plaintext m as input and then outputs a ciphertext C with randomness ρ, where it outputs $\perp$ if $M \notin \mathcal{M}_{ek}$; (3) $m \leftarrow \mathrm{Dec}(dk, C)$: The decryption algorithm takes a decryption key dk and a ciphertext C as input and then outputs a plaintext m; We say that a public key encryption scheme PKE satisfies the correctness if, for any m and λ, $\Pr\left[\mathrm{Dec}(dk, \mathrm{Enc}(ek, m; \rho)) = m\right] = 1$ holds for any $(ek, dk) \leftarrow \mathrm{KeyGen}(1^\lambda)$.

Definition C.1 (IND-CPA). We say that a public key encryption scheme PKE satisfies indistinguishability against chosen-plaintext attacks (IND-CPA) if, for any probabilistic polynomial time adversary $\mathcal{A}$ against the game shown in Fig. 15, the probability $\mathrm{Adv}^{\mathrm{CPA}}_{\mathrm{PKE},\mathcal{A}}(\lambda) := \left|\Pr[\mathrm{CPA}^{\mathcal{A},0}_{\mathrm{PKE}}(\lambda) = 1] - \Pr[\mathrm{CPA}^{\mathcal{A},1}_{\mathrm{PKE}}(\lambda) = 1]\right|$ is negligible for a security parameter λ and $b \leftarrow \{0,1\}$.

$\mathrm{CPA}^{\mathcal{A},b}_{\mathrm{PKE}}(\lambda)$
$(ek, dk) \leftarrow \mathrm{PKE.KeyGen}(1^\lambda)$
$b' \leftarrow \mathcal{A}^{\mathrm{ENC}}(ek)$
if $b = b'$ **then return** 1
else return 0
$\mathrm{ENC}(M_0, M_1)$
$C \leftarrow \mathrm{PKE.Enc}(ek, M_b)$
return C

Figure 15: The IND-CPA game $\mathrm{CPA}^{\mathcal{A},b}_{\mathrm{PKE}}(\lambda)$.

C.3 Non-Interactive Zero-Knowledge Arguments

A non-interactive zero-knowledge argument scheme NArg[R] with respect to a relation $\mathrm{R} : \{0,1\}^* \times \{0,1\}^* \times \{0,1\}^* \rightarrow \{0,1\}$ consists of the following algorithms: (1) $par_{\mathrm{R}} \leftarrow \mathrm{Rel}(1^\lambda)$: the relation parameter generation algorithm takes a security parameter 1^λ as input and then outputs a relation parameter par_{R}, where we assume that 1^λ is efficiently obtained from par_{R} and $\mathbb{L}_{par_{\mathrm{R}}}$ is an NP language; (2) $(\mathrm{crs}, \tau) \leftarrow \mathrm{Setup}(par_{\mathrm{R}})$: the setup algorithm takes a relation parameter par_{R} as input and outputs a common reference string crs and a trapdoor τ, where we assume that par_{R} is efficiently obtained from crs; (3) $\pi \leftarrow \mathrm{Prove}(\mathrm{crs}, \theta, w)$: the prover algorithm takes a common reference string crs, a statement θ, and a witness w as input and outputs a proof π; (4) $b \leftarrow \mathrm{Verify}(\mathrm{crs}, \theta, \pi)$: the verification algorithm takes a common reference string crs, a statement θ, and a proof π as input and outputs $b = 1$ or $b = 0$; and (5) $\pi \leftarrow \mathrm{SimProve}(\mathrm{crs}, \tau, \theta)$: the simulation algorithm takes a common reference string crs, a trapdoor τ, and a statement θ as input and outputs a proof π. We say that a non-interactive zero-knowledge argument scheme NArg[R] satisfies correctness if, for any λ and probabilistic polynomial time adversary $\mathcal{A}$, $\Pr\left[\mathrm{R}\left(\mathrm{Setup}(par_{\mathrm{R}}), \theta, w\right) = 0 \vee \mathrm{Verify}(\mathrm{crs}, \theta, \mathrm{Prove}(\mathrm{crs}, \theta, w)) = 1\right] = 1$ holds for any $par_{\mathrm{R}} \leftarrow \mathrm{Rel}(1^\lambda)$, where $(\mathrm{crs}, \tau) \leftarrow \mathrm{Setup}(par_{\mathrm{R}})$, $par_{\mathrm{R}} \leftarrow \mathrm{Rel}(1^\lambda)$, and $(\theta, w) \leftarrow \mathcal{A}$.

Definition C.2 (Zero-Knowledge). We say that a non-interactive zero-knowledge argument scheme NArg[R] satisfies computational

zero-knowledge if, for any probabilistic polynomial time adversary $\mathcal{A}$ against the game shown in Fig. 16, a probability $\mathrm{Adv}^{\mathrm{ZK}}_{\mathrm{NArg[R]},\mathcal{A}}(\lambda) := \left|\Pr[\mathrm{ZK}^{\mathcal{A},0}_{\mathrm{NArg[R]}}(\lambda)] - \Pr[\mathrm{ZK}^{\mathcal{A},1}_{\mathrm{NArg[R]}}(\lambda)]\right|$ is negligible for a security parameter λ and $b \leftarrow \{0,1\}$.

$\mathrm{ZK}^{\mathcal{A},b}_{\mathrm{NArg[R]}}(\lambda)$
$par_{\mathrm{R}} \leftarrow \mathrm{NArg.Rel}(1^\lambda)$; $(crs, \tau) \leftarrow \mathrm{NArg.Setup}(par_{\mathrm{R}})$
$b' \leftarrow \mathrm{A}^{\mathrm{PROVE}}(crs)$
if $b = b'$ **then return** 1 **else return** 0
$\mathrm{PROVE}(\theta, w)$
if $\mathrm{R}(par_{\mathrm{R}}, \theta, w) = 0$ **then return** $\perp$
$\pi_0 \leftarrow \mathrm{NArg.Prove}(crs, \theta, w)$; $\pi_1 \leftarrow \mathrm{NArg.SimProve}(crs, \tau, \theta)$
return π_b

Figure 16: The zero knowledge game $\mathrm{ZK}^{\mathcal{A},b}_{\mathrm{NArg[R]}}(\lambda)$.

Definition C.3 (Soundness). We say that a non-interactive zero-knowledge argument scheme NArg[R] satisfies computational soundness if the probability $\mathrm{Adv}^{\mathrm{SND}}_{\mathrm{NArg[R]},\mathcal{A}}(\lambda) := \Pr\left[\mathrm{SND}^{\mathcal{A}}_{\mathrm{NArg[R]}}(\lambda) = 1\right]$ that $\mathcal{A}$ wins the game SND shown in Fig. 17 is negligible for a security parameter λ and $b \leftarrow \{0,1\}$.

$\mathrm{SND}^{\mathcal{A}}_{\mathrm{NArg[R]}}(\lambda)$
$par_{\mathrm{R}} \leftarrow \mathrm{NArg.Rel}(1^\lambda)$; $(crs, \tau) \leftarrow \mathrm{NArg.Setup}(par_{\mathrm{R}})$
$(\theta, \pi) \leftarrow \mathcal{A}(crs)$
if $(\mathrm{NArg.Verify}(crs, \theta, \pi) = 1 \wedge \forall w \in \{0,1\}^* :$
$\mathrm{R}(par_{\mathrm{R}}, \theta, w) = 0)$ **then return** 1 **else return** 0

Figure 17: The soundness game $\mathrm{SND}^{\mathcal{A}}_{\mathrm{NArg[R]}}$.

C.4 Blind Signatures

We formally define the security of a BS scheme with weak blindness. We consider a scheme that generates signatures through N_s-round interactions with predicate constructions. We describe the weak blindness and the strong unforgeability.

Definition C.4 (Strong Unforgeability). We say that a PBS scheme satisfies strong unforgeability against chosen message attacks if, for any probabilistic polynomial algorithm $\mathcal{A}$, the probability $\mathrm{Adv}^{\mathrm{UNF}}_{\mathrm{PBS},\mathcal{A}}(\lambda) = \Pr[\mathrm{UNF}^{\mathcal{A}}_{\mathrm{PBS}}(\lambda) = 1]$ that $\mathcal{A}$ wins the game shown in Fig. 18 is negligible for a security parameter λ, where $\mathrm{InjF}(\mathcal{S}, \mathcal{T})$ in the figure is a family of injective functions from a set $\mathcal{S}$ to a set $\mathcal{T}$. $\mathrm{P} : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}$ in the figure is a predicate compiler which takes a predicate prd and a message m and outputs 1 if m satisfies prd and 0 otherwise.

D Remaining Security Definitions of PBAS

We define adaptability, weak blindness, witness hiding, extractability, one-more unforgeability and witness extractability as the remaining security notions of BASes. The following definitions are a part of our first contribution, although they are described below due to the page limitation of the main body.

$\mathrm{UNF}^{\mathcal{A}}_{\mathrm{PBS}}(\lambda)$
$par \leftarrow \mathrm{PBS.Setup}(1^\lambda)$; $(sk, vk) \leftarrow \mathrm{PBS.KeyGen}(par)$
$\vec{W} := [\,]$; $\vec{PRD} := [\,]$
$(m^*_i, \sigma^*_i)_{i\in[n]} \leftarrow \mathcal{A}^{\mathrm{Sign}}(vk)$
if $(n > 0$
$\wedge\ \forall i \in [n] : \mathrm{PBS.Verify}(vk, m^*_i, \sigma^*_i) = 1$
$\wedge\ \forall i \neq j \in [n] : (m^*_i, \sigma^*_i) \neq (m^*_j, \sigma^*_j)$
$\wedge\ \nexists f \in \mathrm{InjF}([n], [|\vec{PRD}|]) : \forall i \in [n] :$
$\mathrm{P}(\vec{PRD}_{f(i)}, m^*_i) = 1)$ **then return** 1
else return 0
$\mathrm{Sign}(prd, msg, i, j)$
if $\vec{W}_i = \perp$ **then return** $\perp$
if $j = 1$ **then**
$(msg', state) \leftarrow \mathrm{PBS.Sign}_1(sk, prd, msg)$
$\vec{W}_i = \vec{W}_i \,||\, (state, prd)$
return msg'
if $j \in [2, N_s - 1]$ **then**
$(state, prd) := \vec{W}_i$
$(msg', state) \leftarrow \mathrm{PBS.Sign}_j(state, msg)$
$\vec{W}_i = \vec{W}_i \,||\, (state, prd)$
return msg'
if $j = N_s$ **then**
$(state, prd) := \vec{W}_i$
$(msg', b) \leftarrow \mathrm{PBS.Sign}_{N_s}(state, msg)$
if $b = 1$ **then**
$\vec{W}_i = \perp$; $\vec{PRD} = \vec{PRD} \,||\, prd$
return msg'
return $\perp$

Figure 18: The strong unforgeability game $\mathrm{UNF}^{\mathcal{A}}_{\mathrm{PBS}}$ for PBS.

$\mathrm{Adapt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda)$
$par \leftarrow \mathrm{PBAS.Setup}(1^\lambda)$; $(sk, vk) \leftarrow \mathrm{PBAS.KeyGen}(par)$
$(m, \hat{\sigma}, (Y, y)) \leftarrow \mathcal{A}(vk)$
if $(Y, y) \notin \mathrm{R} \vee \mathrm{PBAS.pVerify}(vk, m, \hat{\sigma}, Y) = 0$ **then return** $\perp$
if $\mathrm{PBAS.Verify}(\mathrm{PBAS.Adapt}(vk, \hat{\sigma}, y), m, vk) = 1$ **then return** 1
else return 0

Figure 19: The adaptability game $\mathrm{Adapt}^{\mathcal{A}}_{\mathrm{PBAS}}$ for PBAS

The adaptability guarantees that, for any statement Y, it is possible to generate an adapted signature from any pre-signature by using a witness y such that $(Y, y) \in \mathrm{R}$.

Definition D.1 (Adaptability). We say that a PBAS scheme satisfies adaptability if, for any probabilistic polynomial time adversary $\mathcal{A}$, a probability $\mathrm{Adv}^{\mathrm{Adapt}}_{\mathrm{PBAS},\mathcal{A}}(\lambda) = 1 - \Pr[\mathrm{Adapt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda) = 1]$ is negligible for a security parameter λ in Fig. 19

Definition D.2 (Weak Blindness). We say that a PBAS scheme satisfies weak blindness if, for any probabilistic polynomial time

adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, a probability

$$\mathrm{Adv}^{\mathrm{wBLD}}_{\mathrm{PBAS},\mathcal{A}}(\lambda) := \left|\Pr[\mathrm{wBLD}^{\mathcal{A},1}_{\mathrm{PBAS}}(\lambda)] - \Pr[\mathrm{wBLD}^{\mathcal{A},0}_{\mathrm{PBAS}}(\lambda)]\right|$$

that $\mathcal{A}$ wins the game shown in Fig. 20 is negligible for a security parameter λ and $b \leftarrow \{0,1\}$.

1: $\mathrm{wBLD}^{\mathcal{A},b}_{\mathrm{PBAS}}(\lambda)$
2: $par \leftarrow \mathrm{PBAS.Setup}(1^\lambda)$; $(prd, m_0, m_1, vk) \leftarrow \mathcal{A}_1(par)$
3: **if** $\exists i \in \{0,1\} : \mathrm{P}(prd, m_i) = 0$ **then return** $\bot$
4: $W := \emptyset$
5: $b' \leftarrow \mathcal{A}_2^{\mathrm{USER}_0, \mathrm{USER}}(vk)$
6: **return** b'
7: $\mathrm{USER}_0(m_0, m_1)$
8: **if** $\vec{W} \neq \emptyset$ **then return** $\bot$
9: $(msg, state) \leftarrow \mathrm{PBAS.User}_0(vk, prd, m_b)$
10: $W = \{(state, prd)\}$
11: **return** msg
12: $\mathrm{USER}(msg, j)$
13: **if** $W = \bot \vee W = \emptyset$ **then return** $\bot$
14: **if** $j \in [1, N_p - 1]$ **then**
15: $(state, prd) := W$
16: $(msg', state) \leftarrow \mathrm{PBAS.User}_j(state, msg)$
17: $W = \{(state, prd)\}$
18: **return** msg'
19: **if** $j = N_p$ **then**
20: $(state, prd) := W$; $\hat{\sigma} \leftarrow \mathrm{PBAS.User}_{N_p}(msg, state)$
21: **if** $\hat{\sigma} \neq \bot$ **then**
22: $W = \bot$
23: **return** $\bot$
24: **return** $\hat{\sigma}$
25: **return** $\bot$

Figure 20: The weak blindness game $\mathrm{wBLD}^{\mathcal{A},b}_{\mathrm{PBAS}}$ for PBAS.

In Fig.20, W is initialized as an empty set at Line 4 and records output of $\mathrm{PBAS.User}_j$. When the interaction with the User oracle is completed, the User oracle sets $\bot$ in W and closes the session.

The weak blindness is sufficient for applications such as privacy-preserving atomic swaps [11], which are the original motivation of BASes. According to Qin et al. [11], the privacy of users for trading is guaranteed unless a signer knows messages to be signed for given blockchain transactions. One might think that there is a stronger notion of blindness [15] for (non-adaptor) BSes such that instances of the signatures are blinded during the signature generation. However, our definition inherits from the same security notion as Qin et al. with respect to the weak blindness, whereas our definition contains more interfaces during the interaction in the pre-signing algorithm.

The witness hiding guarantees that adapted signatures are indistinguishable from signatures generated by a simulator without knowledge of witnesses. The witness hiding can prevent leakage of the witnesses against an eavesdropper [18].

Definition D.3 (Witness Hiding). We say that a PBAS scheme satisfies witness hiding if, for any probabilistic polynomial time adversary $\mathcal{A}$, there exists a simulator Sim such that the probability $\mathrm{Adv}^{\mathrm{bWh}}_{\mathrm{PBAS},\mathcal{A}}(\lambda) = \left|\Pr[\mathrm{bWh}^{\mathcal{A},1}_{\mathrm{PBAS,Sim}}(\lambda)] - \Pr[\mathrm{bWh}^{\mathcal{A},0}_{\mathrm{PBAS,Sim}}(\lambda)]\right|$ that $\mathcal{A}$ wins the game in Fig. 21 is negligible for a security parameter λ.

The above definition follows a definition by Liu et al. [18]. Although there are further extended definitions [21] of witness hiding, they are based on a functional setting. Our definition can be combined with these extensions, although we leave it as an open problem.

1: $\mathrm{bWh}^{\mathcal{A},b}_{\mathrm{PBAS,Sim}}(\lambda)$
2: $par \leftarrow \mathrm{PBAS.Setup}(1^\lambda)$; $(sk, vk) \leftarrow \mathrm{PBAS.KeyGen}(par)$
3: $(Y, y) \leftarrow \mathrm{R.Gen}(1^\lambda)$
4: **if** $(Y, y) \notin \mathrm{R}$ **then return** $\bot$
5: $b \leftarrow \mathcal{A}^{\mathrm{Chall}_b(\cdot,\cdot,\cdot,\cdot)}(Y)$
6: **return** b
7: $\mathrm{Chall}_0(vk, sk, prd, m)$
8: **if** $\mathrm{P}(prd, m) = 0$ **then return** $\bot$
9: $(b, \hat{\sigma}) \leftarrow \langle \mathrm{PBAS.pSign}(sk, prd, Y), \mathrm{PBAS.User}(vk, prd, m, Y) \rangle$
10: **if** $b = 0$ **then return** $\bot$
11: $\sigma \leftarrow \mathrm{PBAS.Adapt}(vk, \hat{\sigma}, y)$
12: **return** σ
13: $\mathrm{Chall}_1(vk, sk, prd, m)$
14: **if** $\mathrm{P}(prd, m) = 0$ **then return** $\bot$
15: $\sigma \leftarrow \mathrm{Sim}(sk, vk, prd, m, Y)$
16: **return** σ

Figure 21: The witness hiding game $\mathrm{bWh}^{\mathcal{A},b}_{\mathrm{PBAS,Sim}}$ for PBAS.

The extractability guarantees that any probabilistic polynomial-time adversary cannot generate signatures that are unable to extract a witness for any statement used in pre-signatures. This is a weaker security notion of the full extractability in Section 3.2, where the adversary cannot utilize standard signatures for output of the game.

Definition D.4 (Extractability). We say that a predicate blind adaptor signature PBAS scheme satisfies extractability if, for any probabilistic polynomial time adversary $\mathcal{A}$, the probability $\mathrm{Adv}^{\mathrm{bExt}}_{\mathrm{PBAS},\mathcal{A}}(\lambda) = \Pr[\mathrm{bExt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda) = 1]$ that $\mathcal{A}$ wins the game shown in Fig. 22 is negligible for a security parameter λ.

Definition D.5 (One-more Unforgeability). We say that a PBAS scheme satisfies one-more unforgeability if, for any probabilistic polynomial time adversary $\mathcal{A}$, the probability $\mathrm{Adv}^{\mathrm{omaUNF}}_{\mathrm{PBAS},\mathcal{A}}(\lambda) = \Pr[\mathrm{onaUNF}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda) = 1]$ that $\mathcal{A}$ wins the game shown in Fig. 23 is negligible for a security parameter λ.

Definition D.6 (Witness Extractability). We say that a PBAS scheme satisfies witness extractability if, for any probabilistic polynomial time adversary $\mathcal{A}$, the probability

$$\mathrm{Adv}^{\mathrm{baWitExt}}_{\mathrm{PBAS},\mathcal{A}}(\lambda) = \Pr[\mathrm{baWitExt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda) = 1]$$

that $\mathcal{A}$ wins the game shown in Fig. 24 is negligible for a security parameter λ.

$\mathrm{bExt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda)$
$par \leftarrow \mathrm{PBAS.Setup}(1^\lambda)$; $(sk, vk) \leftarrow \mathrm{PBAS.KeyGen}(par)$
$T[m] := \emptyset$ // m is an arbitrary message
$\vec{PRD} := [\,]$; $\vec{W} := [\,]$
$(m_i^*, \sigma_i^*, \{(Y_{i,j}, \hat{\sigma}_{i,j})\})_{i\in[n], j\in[n']} \leftarrow \mathcal{A}^{\mathrm{pSign}}(vk)$
$T[m_i^*] = T[m_i^*] \cup \{(Y_{i,j}, \hat{\sigma}_{i,j})\}$ // $\forall i \in [n]\ \forall j \in [n']$
if $(n > 0$
$\wedge\ \forall i \in [n] : \mathrm{PBAS.Verify}(vk, m_i^*, \sigma_i^*) = 1$
$\wedge\ \forall i \neq j \in [n] : (m_i^*, \sigma_i^*) \neq (m_j^*, \sigma_j^*)$
$\wedge\ \nexists f \in \mathrm{InjF}([n], [|\vec{PRD}|]) : \forall i \in [n] : \mathrm{P}(\vec{PRD}_{f(i)}, m_i^*) = 1$
$\wedge\ \exists i \in [n]\ \exists j \in [n'] : \forall (Y_{i,j}, \hat{\sigma}_{i,j}) \in T[m_i^*] :$
$(Y, \mathrm{PBAS.Ext}(\sigma_i^*, \hat{\sigma}, Y)) \notin \mathrm{R})$ **then return** 1
else return 0
$\mathrm{pSign}(prd, msg, Y, i, j)$
if $\vec{W}_i = \perp$ **then return** $\perp$
if $j = 1$ **then**
  $(msg', state) \leftarrow \mathrm{PBAS.pSign}_1(sk, prd, msg, Y)$
  $\vec{W}_i = \vec{W}_i \,||\, (state, prd)$
  return msg'
if $j \in [2, N_p - 1]$ **then**
  $(state, prd) := \vec{W}_i$
  $(msg', state) \leftarrow \mathrm{PBAS.pSign}_j(state, msg)$
  $\vec{W}_i = \vec{W}_i \,||\, (state, prd)$
  return msg'
if $j = N_p$ **then**
  $(state, prd) := \vec{W}_i$
  $(msg', b) \leftarrow \mathrm{PBAS.pSign}_{N_p}(state, msg)$
  if $b = 1$ **then**
    $\vec{W}_i = \perp$; $\vec{PRD} = \vec{PRD} \,||\, prd$
  return msg'
return $\perp$

Figure 22: The extractability game $\mathrm{bExt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda)$ **for** PBAS**.**

E Details of Security Gaps

We show a gap where the existing scheme by Qin et al. [11] does not satisfy the unlinkability as our technical difficulty. We also show that the existing conversion [14] of the state-of-the-art BSes [15] does not satisfy the full extractability. We note that these gaps do not mean that these schemes are insecure under their original security definitions because our discussion is out of the scope of their work.

Security Gap in the Scheme by Qin et al. We show that the scheme by Qin et al. does not satisfy the unlinkability. They proposed a BAS scheme based on ECDSA. Since the unlinkability requires adapted signatures output by Adapt to be indistinguishable from standard signatures, signatures of the BAS scheme by Qin et al. are identical to the ECDSA signatures. However, the signatures are different for the signature component r with randomness from ECDSA, whose signature components consist of (s, r). The computation of r for the scheme by Qin et al. contains a statement $Y \in \mathbb{G}$ even for the input of Verify while the original signature component r of ECDSA does not contain it. In other words, when the original verification algorithm of ECDSA is executed, it returns 0 for the adapted signatures of the

$\mathrm{omaUNF}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda)$
$par \leftarrow \mathrm{PBAS.Setup}(1^\lambda)$; $(sk, vk) \leftarrow \mathrm{PBAS.KeyGen}(par)$
$\vec{U} := [\,]$; $\vec{W} := [\,]$; $\vec{PRD} := [\,]$; $k_p = 0$
$(Y, y) \leftarrow \mathrm{R.Gen}(1^\lambda)$
$(m_i^*, \sigma_i^*)_{i\in[n]} \leftarrow \mathcal{A}^{\mathrm{pSignChl,Sign,pSign}}(vk)$
if $(n > 0$
$\wedge\ \forall i \in [n] : \mathrm{PBAS.Verify}(vk, m_i^*, \sigma_i^*) = 1$
$\wedge\ \forall i \neq j \in [n] : (m_i^*, \sigma_i^*) \neq (m_j^*, \sigma_j^*)$
$\wedge\ \nexists f \in \mathrm{InjF}([n], |\vec{PRD}|) : \forall i \in [n] : \mathrm{P}(\vec{PRD}_{f(i)}, m_i^*) = 1)$
then return 1
else return 0
$\mathrm{pSignChl}(m^*, i, j)$
if $k_p \geq n \vee \vec{U}_i = \perp$ **then return** $\perp$
if $j = 1$ **then**
  $(msg, state) \leftarrow \mathrm{PBAS.pSign}_1(sk, prd, m^*, Y)$
  $\vec{U}_i = \vec{U}_i \,||\, (state, prd, Y)$
  return (msg, Y)
if $j \in [2, N_p - 1]$ **then**
  $(state, prd, Y) := \vec{U}_i$
  $(msg, state) \leftarrow \mathrm{PBAS.pSign}_j(state, m^*)$
  $\vec{U}_i = \vec{U}_i \,||\, (state, prd, Y)$
  return (msg, Y)
if $j = N_p$ **then**
  $(state, prd, Y) := \vec{U}_i$
  $(msg, b) \leftarrow \mathrm{PBAS.pSign}_{N_p}(state, m^*)$
  if $b = 1$ **then**
    $\vec{U}_i = \perp$; $k_p = k_p + 1$
    return (msg, Y)
return $\perp$
$\mathrm{Sign}(m)$
$\sigma \leftarrow \mathrm{PBAS.Sign}(sk, m)$; **return** σ
$\mathrm{pSign}(prd, msg, Y, i, j)$
if $\vec{W}_i = \perp$ **then return** $\perp$
if $j = 1$ **then**
  $(msg', state) \leftarrow \mathrm{PBAS.pSign}_1(sk, prd, msg, Y)$
  $\vec{W}_i = \vec{W}_i \,||\, (state, prd)$
  return msg'
if $j \in [2, N_p - 1]$ **then**
  $(state, prd) := \vec{W}_i$
  $(msg', state) \leftarrow \mathrm{PBAS.pSign}_j(state, msg)$
  $\vec{W}_i = \vec{W}_i \,||\, (state, prd)$
  return msg'
if $j = N_p$ **then**
  $(state, prd) := \vec{W}_i$
  $(msg', b) \leftarrow \mathrm{PBAS.pSign}_{N_p}(state, msg)$
  if $b = 1$ **then**
    $\vec{W}_i = \perp$; $\vec{PRD} = \vec{PRD} \,||\, prd$
    return msg'
return $\perp$

Figure 23: The one-more unforgeability game $\mathrm{omaUNF}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda)$**.**

$\mathrm{baWitExt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda)$

$par \leftarrow \mathrm{PBAS.Setup}(1^\lambda)$; $(sk, vk) \leftarrow \mathrm{PBAS.KeyGen}(par)$
$\vec{U} := [\,]$; $\vec{W} := [\,]$
$(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathrm{pSignChl,Sign,pSign}}(vk)$
if $(\mathrm{PBAS.Verify}(vk, m^*, \sigma^*) = 0 \vee \mathrm{P}(prd, m^*) = 1)$ **then**
  return $\perp$
$y^* \leftarrow \mathrm{PBAS.Ext}(\sigma^*, \hat{\sigma}^*, Y)$
if $(Y, y^*) \notin \mathrm{R}$ **then return** 1
else return 0

$\mathrm{pSignChl}(m^*, Y, i)$

if $\vec{U}_i = \perp$ **then return** $\perp$
if $i = 1$ **then**
  $(msg, state) \leftarrow \mathrm{PBAS.pSign}_1(sk, prd, m^*, Y)$
  $\vec{U}_i = \vec{U}_i \,||\, (state, prd)$
  return msg
if $i \in [2, N_p - 1]$ **then**
  $(state, prd) := \vec{U}_i$
  $(msg, state) \leftarrow \mathrm{PBAS.pSign}_{\mathrm{i}}(state, m^*)$
  $\vec{U}_i = \vec{U}_i \,||\, (state, prd)$
  return msg
return $\perp$

$\mathrm{Sign}(m)$

$\sigma \leftarrow \mathrm{PBAS.Sign}(sk, m)$; **return** σ

$\mathrm{pSign}(prd, msg, Y, i, j)$

if $\vec{W}_i = \perp$ **then return** $\perp$
if $j = 1$
  $(msg', state) \leftarrow \mathrm{PBAS.pSign}_1(sk, prd, msg, Y)$
  $\vec{W}_i = \vec{W}_i \,||\, (state, prd)$
  return msg'
if $j \in [2, N_p - 1]$ **then**
  $(state, prd) := \vec{W}_i$
  $(msg', state) \leftarrow \mathrm{PBAS.pSign}_{\mathrm{j}}(state, msg)$
  $\vec{W}_i = \vec{W}_i \,||\, (state, prd)$
  return msg'
if $j = N_p$ **then**
  $(state, prd) := \vec{W}_i$
  $(msg', b) \leftarrow \mathrm{PBAS.pSign}_{\mathrm{N_p}}(state, msg)$
  if $b = 1$ **then**
    $\vec{W}_i = \perp$; **return** msg'
return $\perp$

Figure 24: The witness extractability game $\mathrm{baWitExt}^{\mathcal{A}}_{\mathrm{PBAS}}(\lambda)$**.**

scheme by Qin et al., and therefore, an adversary can distinguish the adapted signatures from ECDSA. Consequently, the scheme by Qin et al. does not satisfy the unlinkability. We note that the above result for the unlinkability is out of the scope of their original work.

Security Gap in the Conversion of the Scheme by Fuchsbauer et al. The second gap is on the existing conversion from BSes to ASes: in particular, when we convert the state-of-the-art BS scheme by Fuchsbauer et al. [15] with the existing conversion [14], the resulting scheme does not satisfy the full extractability. In the scheme by Fuchsbauer et al., a user generates additional random numbers $(\alpha, \beta) \in \mathbb{Z}_q^2$ and a statement $Y \in \mathbb{G}$, and then adds them into components (R, s) of the Schnorr signatures to blind the signatures themselves against a signer, i.e., $s' := s + \beta$ and $R' = R + \beta G + \alpha \cdot vk$ where G is a generator of $\mathbb{G}$. These signature components are converted into $s' := s+\beta+y$ and $R' = R+\beta G+\alpha\cdot vk+Y$ with respect to a witness y and a statement Y. The signer is then unable to extract the witness y from the message component s', because the signer knows neither β nor m. This situation enables the user to generate malicious pre-signatures whose witnesses are no longer output from Ext and then store valid signatures in blockchains. Therefore, the above construction does not satisfy the full extractability. We emphasize that the full extractability was not within their original scope.

F Security Proofs of PBASch

We prove that the proposed predicate blind adaptor Schnorr signature scheme PBASch defined in Fig.7 satisfies the security definitions from Section 3.2 and Appendix D. As the proof of the full extractability is shown in Section 5.2, we prove the remaining theorems below.

F.1 Proof of Theorem 5.1

A pre-signature $\hat{\sigma}$ generated through the interaction between a signer and a user is $\hat{\sigma} = (R', s)$, where $R' = rG + Y + \alpha X$ and $s = r + \mathrm{H}(R', X, m)x$. Then, since $sG + Y = (r + (\mathrm{H}(R', X, m) + \alpha)x)G + Y = rG + Y + \mathrm{H}(R', X, m)X + \alpha X = R' + \mathrm{H}(R', X, m)X$ holds, b_1 is 1. Next, the adapted signature σ is $\sigma = (R', s')$, where $s' = s + y$. Therefore, $s'G = (s+y)G = sG + yG = R' + \mathrm{H}(R', X, m)X$ holds, and so b_2 is 1. Finally, the extracted y computed by $s' - s$ satisfies $yG = Y$, therefore $(Y, y) \in \mathrm{R}$. Then, b_3 is 1. Thus, PBASch satisfies the correctness. □

F.2 Proof of Theorem 5.2

Let $\hat{\sigma}$ and (Y, y) be such that $(Y, y) \in \mathrm{R}$ and $1 \leftarrow \mathrm{PBASch.pVerify}(vk, m, \hat{\sigma}, Y)$. This means that $Y = yG$ and $sG + Y = R' + \mathrm{H}(R', X, m)X$ holds. From $(R', s' = s+y) := \sigma \leftarrow \mathrm{PBASch.Adapt}(vk, \hat{\sigma}, y)$, the following holds: $s'G = (s+y)G = sG + Y = R' + \mathrm{H}(R', X, m)X$. Therefore, $\mathrm{PBASch.Verify}(\sigma, m, vk) = 1$ holds, and therefore PBASch satisfies the pre-signature adaptability. □

F.3 Proof of Theorem 5.3

We give a formal proof that PBASch scheme satisfies the weak blindness by providing reductions to the security properties of its building blocks. We proceed by a sequence of games in Fig. 26.

G_0: In Fig.26, G_0 is the game where all colored boxes are ignored. The game is the blindness game in Fig.20 for PBASch.

G_1: G_1 is obtained by adding the processing in the light green box to G_0. The game was modified to return a proof generated by the NArg.SimProve algorithm instead of one generated by the NArg.Prove algorithm. We show that the upper bound of the adversary's advantage due to the change from G_0 to G_1 is the ZK game for NArg in Fig.16.

G_2: G_2 is obtained by adding the processing in the orange box to G_1. In G_2, a fixed message $\overline{m}$ is encrypted instead of m_b. We show that the upper bound of the adversary's advantage due to the change from G_1 to G_2 is the IND-CPA game for PKE in Fig.15.

1: nfPBAS.Setup(1^λ)
2: $par \leftarrow$ PBASch.Setup(1^λ)
3: **return** par
4: nfPBAS.KeyGen(par)
5: $(sk, vk) \leftarrow$ PBASch.KeyGen(par)
6: **return** (sk, vk)
7: nfPBAS.User$_0(vk, prd, m)$
8: $(msg_{U,0}, state_{U,0}) \leftarrow$ PBASch.User$_0(vk, prd, m)$
9: **return** $(msg_{U,0}, state_{U,0})$
10: nfPBAS.pSign$_1(sk, prd, msg_{U,0}, Y)$
11: $(msg_{S,1}, state_S) \leftarrow$ PBASch.pSign$_1(sk, prd, msg_{U,0}, Y^*)$
12: **return** $(msg_{S,1}, state_S)$
13: nfPBAS.User$_1(state_{U,0}, msg_{S,1})$
14: $(msg_{U,1}, state_{U,1}) \leftarrow$ PBASch.User$_1(state_{U,0}, msg_{S,1})$
15: **return** $(msg_{U,1}, state_{U,1})$
16: nfPBAS.pSign$_2(state_S, msg_{U,1})$
17: $(vk, prd, C, r, \hat{R}) := state_S$; $(c, \pi) := msg_{U,1}$
1: $(msg'_{S,2}, b) \leftarrow$ PBASch.pSign$_2(state_S, msg_{U,1})$
2: $s = r + cx$; $\sigma = (rG, s)$ // Sch.Sign
3: $C_0 := \mathrm{F}(sk, c)$; $C_1 := C_0 \oplus \sigma$
4: $b' \leftarrow \{0,1\}$; $msg_{S,2} := (msg'_{S,2}, C_{b'})$
5: **return** $(msg_{S,2}, b)$
6: nfPBAS.User$_2(state_{U,1}, msg_{S,2})$
7: $(msg'_{S,2}, C) := msg_{S,2}$
8: $\hat{\sigma}' \leftarrow$ PBASch.User$_2(state_{U,1}, msg'_{S,2})$
9: $\hat{\sigma} := (\hat{\sigma}', C)$
10: **return** $\hat{\sigma}$
11: nfPBAS.Adapt$(vk, \hat{\sigma}, y)$
12: $(\hat{\sigma}', C) := \hat{\sigma}$
13: $\sigma \leftarrow$ PBASch.Adapt$(vk, \hat{\sigma}', y)$
14: **return** σ
15: nfPBAS.Ext$(\sigma, \hat{\sigma}, Y)$
16: $(\hat{\sigma}', C) := \hat{\sigma}$
17: $y \leftarrow$ PBASch.Ext$(\sigma, \hat{\sigma}', Y)$
18: **return** y

Figure 25: The predicate blind adaptor signature scheme nfPBAS**.** F **is a secure PRF.**

G_0 and G_1. Consider the probabilistic polynomial time adversary Z_b playing the zero knowledge game for NArg[$\mathrm{R}_{\mathrm{Sch}}$] as shown in Fig.27. By the definitions of each game, we have $\Pr[\mathrm{ZK}^{\mathrm{Z}_b,0}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}]}] = \Pr[\mathrm{G}_0^{\mathcal{A},b}] = \Pr[\mathrm{wBLD}^{\mathcal{A},b}_{\mathrm{PBASch}}]$, $\Pr[\mathrm{ZK}^{\mathrm{Z}_b,1}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}]}] = \Pr[\mathrm{G}_1^{\mathcal{A},b}]$, and therefore

$$\begin{aligned}\mathrm{Adv}^{\mathrm{ZK}}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}],\mathrm{Z}_b} &:= \left|\Pr[\mathrm{ZK}^{\mathrm{Z}_b,0}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}]}] - \Pr[\mathrm{ZK}^{\mathrm{Z}_b,1}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}]}]\right| \\ &= \left|\Pr[\mathrm{wBLD}^{\mathcal{A},b}_{\mathrm{PBASch}}] - \Pr[\mathrm{G}_1^{\mathcal{A},b}]\right|.\end{aligned}$$

By the above equation, we have the following equation:

$$\begin{aligned}&\mathrm{Adv}^{\mathrm{wBLD}}_{\mathrm{PBASch},\mathcal{A}} := \left|\Pr[\mathrm{wBLD}^{\mathcal{A},1}_{\mathrm{PBASch}}] - \Pr[\mathrm{wBLD}^{\mathcal{A},0}_{\mathrm{PBASch}}]\right| \\ &= \Big|\Pr[\mathrm{wBLD}^{\mathcal{A},1}_{\mathrm{PBASch}}] - \Pr[\mathrm{G}_1^{\mathcal{A},1}] + \Pr[\mathrm{G}_1^{\mathcal{A},1}] \\ &\quad - \Pr[\mathrm{wBLD}^{\mathcal{A},0}_{\mathrm{PBASch}}] - \Pr[\mathrm{G}_1^{\mathcal{A},0}] + \Pr[\mathrm{G}_1^{\mathcal{A},0}]\Big| \\ &\le \mathrm{Adv}^{\mathrm{ZK}}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}],\mathrm{Z}_1} + \left|\Pr[\mathrm{G}_1^{\mathcal{A},1}] - \Pr[\mathrm{G}_1^{\mathcal{A},0}]\right| + \mathrm{Adv}^{\mathrm{ZK}}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}],\mathrm{Z}_0}\end{aligned} \tag{16}$$

G_1 and G_2. Consider the probabilistic polynomial time adversary C_b playing the IND-CPA game for PKE as shown in Fig.28. By the definitions of each game, we have $\Pr[\mathrm{CPA}^{\mathrm{C}_b,0}_{\mathrm{PKE}}] = \Pr[\mathrm{G}_2^{\mathcal{A},b}]$, $\Pr[\mathrm{CPA}^{\mathrm{C}_b,1}_{\mathrm{PKE}}] = \Pr[\mathrm{G}_1^{\mathcal{A},b}]$, and therefore

$$\begin{aligned}\mathrm{Adv}^{\mathrm{CPA}}_{\mathrm{PKE},\mathrm{C}_b} &:= \left|\Pr[\mathrm{CPA}^{\mathrm{C}_b,1}_{\mathrm{PKE}}] - \Pr[\mathrm{CPA}^{\mathrm{C}_b,0}_{\mathrm{PKE}}]\right| \\ &= \left|\Pr[\mathrm{G}_1^{\mathcal{A},b}] - \Pr[\mathrm{G}_2^{\mathcal{A},b}]\right|.\end{aligned}$$

By the above equation, we have the following equation:

$$\begin{aligned}\left|\Pr[\mathrm{G}_1^{\mathcal{A},1}] - \Pr[\mathrm{G}_1^{\mathcal{A},0}]\right| &= \Big|\Pr[\mathrm{G}_1^{\mathcal{A},1}] - \Pr[\mathrm{G}_2^{\mathcal{A},1}] + \Pr[\mathrm{G}_2^{\mathcal{A},1}] \\ &\quad + \Pr[\mathrm{G}_2^{\mathcal{A},0}] - \Pr[\mathrm{G}_2^{\mathcal{A},0}] - \Pr[\mathrm{G}_1^{\mathcal{A},0}]\Big| \\ &\le \mathrm{Adv}^{\mathrm{CPA}}_{\mathrm{PKE},\mathrm{C}_1} + \left|\Pr[\mathrm{G}_2^{\mathcal{A},1}] - \Pr[\mathrm{G}_2^{\mathcal{A},0}]\right| \\ &\quad + \mathrm{Adv}^{\mathrm{CPA}}_{\mathrm{PKE},\mathrm{C}_0}.\end{aligned} \tag{17}$$

$\mathrm{G}_2^{\mathcal{A},0}$ and $\mathrm{G}_2^{\mathcal{A},1}$. In $\mathrm{G}_2^{\mathcal{A},b}$, the ciphertext C is an encryption of a fixed message $M := \overline{m}$. Moreover, the proof π is output by the simulator NArg.SimProve. Therefore, C and π contain no information about b, so no information about b can be obtained from the pre-signature $\hat{\sigma} = (R', s)$. Consequently, we have the following equation:

$$\left|\Pr[\mathrm{G}_2^{\mathcal{A},0}] - \Pr[\mathrm{G}_2^{\mathcal{A},1}]\right| = 0. \tag{18}$$

Finally, from Equations (16), (17), (18), the advantage of the blindness game for PBASch is as follows:

$$\begin{aligned}\mathrm{Adv}^{\mathrm{wBLD}}_{\mathrm{PBASch},\mathcal{A}} &\le \mathrm{Adv}^{\mathrm{ZK}}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}],\mathrm{Z}_0} + \mathrm{Adv}^{\mathrm{ZK}}_{\mathrm{NArg}[\mathrm{R}_{\mathrm{Sch}}],\mathrm{Z}_1} \\ &\quad + \mathrm{Adv}^{\mathrm{CPA}}_{\mathrm{PKE},\mathrm{C}_0} + \mathrm{Adv}^{\mathrm{CPA}}_{\mathrm{PKE},\mathrm{C}_1}.\end{aligned}$$

This completes the proof of Theorem 5.3. □

F.4 Proof of Theorem 5.4

We give a formal proof that our proposed scheme PBASch as defined in Fig.7 satisfies the unlinkability. This is achieved through reductions to the security properties of its building blocks. We proceed with game transformations specified in Fig.29.

G_0 : In Fig.29, G_0 is the game where all colored boxes are ignored. The game is the unlinkability game in Fig.5 of PBASch.

G_1 : G_1 is obtained by adding the processing in the light green box to G_0. In this game, we generate a random pair consisting of a statement and witness (Y', y'), which are then utilized to generate both a

$\underline{\text{wBLD}^{\mathcal{A},b}_{\text{PBASch}}(\lambda),\ \text{G}^{\mathcal{A},b}_0,\ \text{G}^{\mathcal{A},b}_1,\ \text{G}^{\mathcal{A},b}_2}$

$(q, \mathbb{G}, G, \text{H}) = sp \leftarrow \text{Sch.Setup}(1^\lambda)$
$(crs, \tau) \leftarrow \text{NArg.Setup}(sp);\ (ek, dk) \leftarrow \text{PKE.KeyGen}(1^\lambda)$
$(Y, y) \leftarrow \text{R.Gen}(1^\lambda)$
$(prd, m_0, m_1, vk) \leftarrow \mathcal{A}_1(crs, ek)$
if $\exists i \in \{0,1\} : \text{P}(prd, m_i) = 0$ **then return** 0
$W := \emptyset;\ \rho \leftarrow \mathcal{R}_{ek}$
$b' \leftarrow \mathcal{A}_2^{\text{USER}_0, \text{USER}}(vk)$
return b'

$\underline{\text{USER}_0(m_0, m_1)}$
if $W \neq \emptyset$ **then return** $\perp$
$M := m_b$
$M := \overline{m}$ // $\overline{m}$ is a fixed message
$C := \text{PKE.Enc}(ek, M; \rho)$
$state := (ck, prd, M, \rho, C);\ W = \{(state, prd)\}$
return C

$\underline{\text{USER}(msg, j)}$
if $W = \perp \vee W = \emptyset$ **then return** $\perp$
if $j = 1$ **then**
  $\hat{R} := msg;\ (state, prd) := W$
  $\alpha \leftarrow \mathbb{Z}_q;\ R' = \hat{R} + \alpha X;\ c := \text{H}(R', X, M) + \alpha$
  $\theta := (X, R', c, C, prd, ek);\ w := (M, \rho.\alpha)$
  $\pi \leftarrow \text{NArg.Prove}(crs, \tau, w)$
  $\pi \leftarrow \text{NArg.SimProve}(crs, \tau, \theta)$
  $state = (vk, R');\ W = \{(state, prd)\}$
  return (c, π)
if $j = 2$ **then**
  $s := msg;\ (state, prd) := W$
  if $sG + Y = R' + \text{H}(R', X, M)X$ **then**
    $W = \perp;\ \hat{\sigma} = (R', s)$
    return $\hat{\sigma}$
return $\perp$

Figure 26: The blindness game for PBASch**.**

$\underline{\text{Z}^{\text{PROVE}}_b(crs)}$
$(q, \mathbb{G}, G, \text{H}) := crs$
$(ek, dk) \leftarrow \text{PKE.KeyGen}(1^\lambda)$
$(Y, y) \leftarrow \text{R.Gen}(1^\lambda)$
$(prd, m_0, m_1, vk) \leftarrow \mathcal{A}_1(crs, ek)$
if $\exists i \in \{0,1\} : \text{P}(prd, m_i) = 0$ **then return** 0
$W := \emptyset;\ \rho \leftarrow \mathcal{R}_{ek}$
$b' \leftarrow \mathcal{A}_2^{\text{USER}_0, \text{USER}}(vk)$
return b'

$\underline{\text{USER}(msg, j)}$
if $W = \perp \vee W = \emptyset$ **then return** $\perp$
if $j = 1$ **then**
  $\hat{R} := msg$
  $\alpha \leftarrow \mathbb{Z}_q;\ R' = \hat{R} + \alpha X$
  $(state, prd) := W$
  $c := \text{H}(R', X, M) + \alpha$
  $\theta := (X, R', c, C, prd, ek)$
  $w := (M, \rho.\alpha)$
  $\pi \leftarrow \text{Prove}(crs, \tau, w)$
  $state = (vk, R')$
  $W = \{(state, prd)\}$
  return (c, π)

Figure 27: Z_b **playing against the zero-knowledge game for** $\text{NArg}[\text{R}_{\text{Sch}}]$**.**

$\underline{\text{C}^{\text{ENC}}_b(ek)}$
$sp \leftarrow \text{Sch.Setup}(1^\lambda)$
$(csr, \tau) \leftarrow \text{NArg.Setup}(sp)$
$(Y, y) \leftarrow \text{R.Gen}(1^\lambda)$
$(prd, m_0, m_1, vk) \leftarrow \mathcal{A}_1(crs, ek)$
if $\exists i \in \{0,1\} : \text{P}(prd, m_i) = 0$ **then return** 0
$W := \emptyset$
$b' \leftarrow \mathcal{A}_2^{\text{USER}_0, \text{USER}}(vk)$
return b'

$\underline{\text{USER}_0(m_0, m_1)}$
if $W \neq \emptyset$ **then return** $\perp$
$M_0 := \overline{m}$
$M_1 := m_b$
$C := \text{Enc}(M_0, M_1)$
$state := (ck, prd, M, \rho, C)$
$W = \{(state, prd)\}$
return C

Figure 28: C_b **playing against the IND-CPA game for** PKE**.**

pre-signature and signature. We show that the upper bound of the adversary's advantage due to the change from G_0 to G_1 is the SRSR_{DL} game in Fig.30.

$\underline{\text{G}_0 \text{ and } \text{G}_1}$. Consider the probabilistic polynomial time adversary $\underline{\text{S}_b}$ defined in Fig.31 against the SRSR_{DL} game in Fig.30. By the definitions of each game, we have $\Pr[\text{G}_0^{\mathcal{A},b}] = \Pr[\text{Unlink}^{\mathcal{A},b}_{\text{PBASch}}]$, $\Pr[\text{SRSR}^{\text{S}_b,1}_{\text{DL}}] = \Pr[\text{G}_1^{\mathcal{A},b}]$, and therefore

$$\begin{aligned}\text{Adv}^{\text{SRSR}}_{\text{DL},\text{S}_b} &:= \left|\Pr[\text{SRSR}^{\text{S}_b,0}_{\text{DL}}] - \Pr[\text{SRSR}^{\text{S}_b,1}_{\text{DL}}]\right| \\ &= \left|\Pr[\text{Unlink}^{\mathcal{A},b}_{\text{PBASch}}] - \Pr[\text{G}_1^{\mathcal{A},b}]\right|.\end{aligned}$$

By the above equation and the triangular inequality, we have:

$$\begin{aligned}\text{Adv}^{\text{Unlink}}_{\text{PBASch},\mathcal{A}} &:= \left|\Pr[\text{Unlink}^{\mathcal{A},1}_{\text{PBASch}}] - \Pr[\text{Unlink}^{\mathcal{A},0}_{\text{PBASch}}]\right| \\ &= \Big|\Pr[\text{Unlink}^{\mathcal{A},1}_{\text{PBASch}}] - \Pr[\text{G}_1^{\mathcal{A},1}] + \Pr[\text{G}_1^{\mathcal{A},1}] \\ &\quad - \Pr[\text{Unlink}^{\mathcal{A},0}_{\text{PBASch}}] - \Pr[\text{G}_1^{\mathcal{A},0}] + \Pr[\text{G}_1^{\mathcal{A},0}]\Big| \\ &\leq \text{Adv}^{\text{SRSR}}_{\text{DL},\text{S}_1} + \left|\Pr[\text{G}_1^{\mathcal{A},1}] - \Pr[\text{G}_1^{\mathcal{A},0}]\right| + \text{Adv}^{\text{SRSR}}_{\text{DL},\text{S}_0} \qquad (19)\end{aligned}$$

$\underline{\text{G}_1^{\mathcal{A},0} \text{ and } \text{G}_1^{\mathcal{A},1}}$. In G_1, σ_0 is an adapted signature. Let $r_0 \leftarrow \mathbb{Z}_q$ be the random number generated in the PBASch.Sign_1 algorithm, then $\sigma_0 = (\hat{R}_0, s')$, where $\hat{R}_0 = r_0 G + Y$ and $s' = r_0 + y + \text{H}(\hat{R}_0, X, m)x)$. Let $r_1 \leftarrow \mathbb{Z}_q$ be the random number generated in the Sch.Sign_1 algorithm, then $\sigma_1 = (R_1, s)$, where $R_1 = r_1 G$ and $s = r_1 + y + \text{H}(\hat{R}_1, X, m)x)$. Let $r' = r_0 + y$. Given fixed values for r_0 and y, r' is uniquely determined. Consequently, the distributions of r' and r_1 are identical. It follows that the distributions of $\sigma_0 = (r'G, r' + \text{H}(\hat{R}, X, m)x)$ and $\sigma_1 = (r_1 G, r_1 + \text{H}(R, X, m)x)$ are also identical. Therefore, we have the following equation:

$$\left|\Pr[\text{G}_1^{\mathcal{A},0}] - \Pr[\text{G}_1^{\mathcal{A},1}]\right| = 0 \qquad (20)$$

Finally, from Equations (19) and (20), the advantage of the unlinkability game for PBASch is as follows:

$$\text{Adv}^{\text{Unlink}}_{\text{PBASch},\mathcal{A}} \leq \text{Adv}^{\text{SRSR}}_{\text{DL},\text{S}_0} + \text{Adv}^{\text{SRSR}}_{\text{DL},\text{S}_1}. \qquad \square$$

F.5 Proof of Theorem 5.6

We present the witness hiding game for PBASch in Fig. 32. Here, the simulator Sim is replaced with the signing algorithm Sch.Sign. Consider the probabilistic polynomial time adversary $\mathcal{A}'_b$ defined in Fig. 33 against the unlinkability game in Fig. 5. The Chall_0 and Chall_1 oracles in the witness hiding game can be simulated

$\underline{\text{Unlink}^{\mathcal{A},b}_{\text{PBASch}}(\lambda)}$, $\underline{\text{G}^{\mathcal{A},b}_0}$, $\underline{\text{G}^{\mathcal{A},b}_1}$
$sp := (q, \mathbb{G}, G, \text{H}) \leftarrow \text{Sch.Setup}(1^\lambda)$
$(crs, \tau) \leftarrow \text{NArg.Setup}(sp)$; $(ek, dk) \leftarrow \text{PKE.KeyGen}(1^\lambda)$
$par := (sp, crs, ek)$; $(sk, vk) \leftarrow \text{PBASch.KeyGen}(par)$
$(prd, m, X, state) \leftarrow \mathcal{A}_1(crs, ek)$
if $\text{P}(prd, m) = 0$ **then return** $\bot$
$\vec{WU} := [\,]$; $\vec{WS} := [\,]$
$b' \leftarrow \mathcal{A}_2^{\text{SignChl,Sign,User,pSign}}(vk)$
if $b = b'$ **then return** 1 **else return** 0
SignChl$(m, prd, (Y, y))$
$y' \leftarrow \mathbb{Z}_q$; $Y' = y'G$; $(Y, y) := (Y', y')$
if $(Y, y) \notin R$ **then return** $\bot$
$(b, \hat{\sigma}) \leftarrow \langle \text{PBASch.pSign}(sk, prd, msg, Y),$
$\quad \text{PBASch.User}(vk, prd, m, Y) \rangle$
if $b = 0$ **then return** $\bot$
$\sigma_0 \leftarrow \text{PBASch.Adapt}(vk, \hat{\sigma}, y)$; $\sigma_1 \leftarrow \text{Sch.Sign}(sk, m)$
return σ_b
Sign(m)
$\sigma \leftarrow \text{PBAS.Sign}(sk, m)$; **return** σ
User(msg, i, j)
if $\vec{WU}_i = \bot$ **then return** $\bot$
if $j = 0$ **then**
  $(msg', state) \leftarrow \text{PBASch.User}_0(vk, prd, msg)$
  $\vec{WU}_i = \vec{WU}_i \,||\, (state, prd)$; **return** msg'
if $j = 1$ **then**
  $(state, prd) := \vec{WU}_i$
  $(msg', state) \leftarrow \text{PBASch.User}_1(state, msg)$
  $\vec{WU}_i = \vec{WU}_i \,||\, (state, prd)$; **return** msg'
if $j = 2$ **then**
  $(state, prd) := \vec{WU}_i$
  $(msg', state) \leftarrow \text{PBASch.User}_2(state, msg)$
  if $\hat{\sigma} \neq \bot$ **then**
    $\vec{WU}_i = \bot$; **return** $\hat{\sigma}$
return $\bot$
pSign(msg, Y, i, j)
if $\vec{WS}_i = \bot$ **then return** $\bot$
if $j = 1$ **then**
  $(msg', state) \leftarrow \text{PBASch.pSign}_1(sk, prd, msg, Y)$
  $\vec{WS}_i = \vec{WS}_i \,||\, (state, prd)$; **return** msg'
if $j = 2$ **then**
  $(state, prd) := \vec{WS}_i$; $(msg', b) \leftarrow \text{PBAS.pSign}_2(state, msg)$
  if $b = 1$ **then**
    $\vec{WS}_i = \bot$; **return** msg'
return $\bot$

Figure 29: The unlinkability game for PBASch**.**

using the SignChl oracle in the unlinkability. Then, if $\mathcal{A}$ wins the witness hiding game, $\mathcal{A}'$ also wins the unlinkability game. Therefore, Equation (15) holds, and Theorem 5.6 is proven. □

$\underline{\text{SRSR}^{\mathcal{A},b}_{\text{DL}}(\lambda)}$
$(q, \mathbb{G}, G) \leftarrow A(1^\lambda)$
$b' \leftarrow \mathcal{A}^{\text{New}}()$
return $b = b'$
New(Y, y)
if $yG \neq Y$ **then return** $\bot$
$y' \leftarrow \mathbb{Z}_q$; $Y' = y'G$
$r \leftarrow \mathbb{Z}_q$; $R = rG$
$Y_0 := Y + rG$; $y_0 := y + r$
$Y_1 := Y' + rG$; $y_1 := y' + r$
return (Y_b, y_b)

Figure 30: The SRSR game for the discrete logarithm relation.

$\underline{\text{S}^{\text{New}}_b(\lambda)}$
$(q, \mathbb{G}, G, \text{H}) \leftarrow \text{PBASch.Setup}(1^\lambda)$; $sp := (q, \mathbb{G}, G, \text{H})$
$(crs, \tau) \leftarrow \text{NArg.Setup}(sp)$; $(ek, dk) \leftarrow \text{PKE.KeyGen}(1^\lambda)$
$(prd, m, X, state) \leftarrow \mathcal{A}_1(crs, ek)$
if $\text{P}(prd, m) = 0$ **then return** $\bot$
$b' \leftarrow \mathcal{A}_2^{\text{SignChl}}(vk)$
return $b = b'$
SignChl$(m, (Y, y))$
$(Y', y') \leftarrow \text{New}(Y, y)$; $(Y, y) := (Y', y')$
if $(Y, y) \notin R$ **then return** $\bot$
$(b, \hat{\sigma}) \leftarrow \langle \text{PBASch.pSign}(sk, prd, msg, Y),$
$\quad \text{PBASch.User}(vk, prd, m, Y) \rangle$
if $b = 0$ **then return** $\bot$
$\sigma_0 \leftarrow \text{PBASch.Adapt}(vk, \hat{\sigma}, y)$; $\sigma_1 \leftarrow \text{Sch.Sign}(sk, m)$
return σ_b

Figure 31: The adversary S **against the** SRSR_{DL} **game.**

$\underline{\text{bWh}^{\mathcal{A},b}_{\text{PBASch,Sch.Sign}}(\lambda)}$
$(q, \mathbb{G}, G, \text{H}) \leftarrow \text{Sch.Setup}(1^\lambda)$
$sp := (q, \mathbb{G}, G, \text{H}))$
$(csr, \tau) \leftarrow \text{NArg.Setup}(sp)$; $(ek, dk) \leftarrow \text{PKE.KeyGen}(1^\lambda)$
$par := (sp, crs, ek)$
$(sk, vk) \leftarrow \text{PBASch.KeyGen}(par)$
$(Y, y) \leftarrow \text{R.Gen}(1^\lambda)$
if $(Y, y) \notin \text{R}$ **then return** $\bot$
$b \leftarrow \mathcal{A}^{\text{Chall}_b(\cdot,\cdot,\cdot,\cdot)}(Y)$
return b
$\text{Chall}_0(vk, sk, prd, m)$
if $\text{P}(prd, m) = 0$ **then return** $\bot$
$(b, \hat{\sigma}) \leftarrow \langle \text{PBASch.pSign}(sk, prd, msg, Y),$
$\quad \text{PBASch.User}(vk, prd, m, Y) \rangle$
if $b = 0$ **then return** $\bot$
$\sigma \leftarrow \text{PBASch.Adapt}(vk, \hat{\sigma}, y)$
return σ
$\text{Chall}_1(vk, sk, prd, m)$
if $\text{P}(prd, m) = 0$ **then return** $\bot$
$\sigma \leftarrow \text{Sch.Sign}(sk, m)$
return σ

Figure 32: The witness hiding game $\text{bWh}^{\mathcal{A},b}_{\text{PBASch,Sch.Sign}}$ **for** PBASch**.**

$\mathcal{A}'^{\text{Chall}_0,\text{Chall}_1}_b(\lambda)$

$par \leftarrow \text{Sch.Setup}(1^\lambda)$

$sp := (q, \mathbb{G}, G, \text{H})$

$(csr, \tau) \leftarrow \text{NArg.Setup}(sp)$; $(ek, dk) \leftarrow \text{PKE.KeyGen}(1^\lambda)$

$par := (sp, crs, ek)$

$(sk, vk) \leftarrow \text{PBASch.KeyGen}(par)$

$(Y, y) \leftarrow \text{R.Gen}(1^\lambda)$

if $(Y, y) \notin R$ **then return** $\perp$

$b' \leftarrow \mathcal{A}^{\text{SignChl}}(vk)$

if $b = b'$ **then return** 1 **else return** 0

$\text{Chall}_b(m, prd, (Y, y))$

if $\text{P}(prd, m) = 0$ **then return** $\perp$

$\sigma_b \leftarrow \text{SignChl}(m, prd, (Y, y))$

return σ_b

Figure 33: The adversary $\mathcal{A}'$ against the unlinkability game for PBASch**, using the adversary $\mathcal{A}$ against the witness hiding game for** PBASch**.**

F.6 Proof of Theorem 5.7

According to Gehart et al. [14], if there exists a membership test for a given statement, an adaptor signature scheme satisfies pre-verify soundness. Our proposed scheme supports efficient membership tests using Koshelev's method [38]. In PBASch.pVerify algorithm, we efficiently verify the language membership of a statement Y using Koshelev's method. If Y is not in the language of the relation, PBASch.pVerify returns $\perp$. Therefore, PBASch satisfies the pre-verify soundness. □